

Démarrage De Conversation Amicale

<https://gemini.google.com/app/cfedc835733cb94c>

User prompt: 1. Je vise une application mobile avec react 2. Les événements seront gérés manuellement 3. Je t'envoierai la capture des fonctionnalités

Response: C'est un excellent choix ! **React Native** (avec **Expo**) est parfaitement adapté pour reproduire ce type d'interface réactive, rapide et dense.

Gérer les événements manuellement au début simplifie grandement l'architecture : pas besoin d'API tierce complexe et payante, tu auras un contrôle total sur tes matchs, cotes et résultats via une base de données simple.

Pour préparer le terrain avant que tu m'envoies les captures :

1. Stack recommandée pour React Native

- **Framework** : Expo (avec le nouveau *Expo Router* ou *React Navigation*).
- **Gestion d'état global (crucial pour le Bet Slip)** : **Zustand** ou **Redux Toolkit** (Zustand est beaucoup plus léger et idéal pour gérer le coupon flottant, le panier de cotes et le solde).
- **Styles / UI** : **Tailwind CSS (NativeWind)** ou **StyleSheet standard**, avec un thème sombre (Dark Mode) dense typique de 1xBet / Melbet.
- **Backend simple pour l'admin** : **Supabase** (PostgreSQL + Auth + API instantanée) ou **Firebase / Node.js (Express/NestJS)**.

2. Modèle de données de base (schéma mental)

Pour que tes matchs manuels fonctionnent, il te faudra structurer :

- Sport (ex: Football, Basketball)
- League / Tournament (ex: Ligue des Champions, Premier League)
- Match (Équipe A vs Équipe B, date/heure, statut : *upcoming, live, finished*)
- Markets & Odds (ex: Marché "1X2" -> 1: 1.85, X: 3.40, 2: 4.20)
- User & Bets (Solde utilisateur, paris validés, calcul des gains)

Dès que tu as les captures sous la main, partage-les ou décris-moi les écrans ! On commencera à découper les composants (composant Carte Match, sélecteur de cotes, tiroir de coupon coulissant, etc.).

User prompt: Après l'interface de connexion on se retrouve sur l'interface (populaire) On retrouve les fonctionnalités - Accéder au studio - Accéder à Apple of fortune - Accéder à Crash - Accéder au studio combat mortal - Accéder au studio TV poker on a aussi l'interface (menu) avec les fonctionnalités - Apple of fortune - Gestion du solde - Création de coupon - Studio Mortal kombat -Studio TV Poker -Crash -Saisie des résultats -Créer championnats & équipes -Apparence -Obtenir votre Assistant gratuitement -Revoir l'onboarding - Se déconnecter on a aussi l'interface (Historique) avec les fonctionnalités Résumé UI/UX complet — Partie « Historique » La section Historique est conçue comme l'espace central permettant à l'utilisateur de consulter, analyser et gérer ses anciens coupons de pari. L'interface doit privilégier la lisibilité, la rapidité d'accès aux informations et des actions contextuelles simples. 1. Écran principal — Historique L'écran comporte un en-tête : • Titre : « Historique des » • Icône de filtre / entonnoir • Icône permettant d'accéder aux options complémentaires • Sélection du compte principal • Solde disponible • Bouton + Effectuer un dépôt Filtrage principaux Deux filtres rapides sont visibles : • Période : par exemple « 1 mois » • Type : par exemple « Vente » Ces filtres permettent de réduire rapidement la liste des opérations affichées. — 2. Statistiques de la période Une zone de synthèse apparaît sous les filtres. Exemple : Statistiques pour la période Paris : 50 · 148338 F Cette zone permet à l'utilisateur d'obtenir immédiatement une vision globale de son activité sur la période sélectionnée. En cliquant dessus, on peut ouvrir une vue plus détaillée des statistiques. — 3. Card d'un pari Chaque coupon est présenté sous forme de carte indépendante. Informations principales En haut de la carte : • Date et heure • Numéro du coupon • Type de pari : Simple / Combiné • Nombre de sélections À droite : • Notification • Menu des actions Informations financières La carte présente ensuite : • Cote • Mise • Gains potentiels • Statut Exemple : Cote : 33 516 Mise : 3 500 F Gains potentiels : 117 306 000 F Statut : ☒ Accepté Le statut doit être facilement identifiable visuellement : Accepté, En cours, Gagné, Perdu, Annulé, etc. — 4. Menu — Actions du coupon Le bouton constitue le centre de gestion du coupon. Lorsqu'il est sélectionné, un Bottom Sheet apparaît. Options disponibles Voir le détail Ouvre la page complète du coupon et de ses événements. Saisir les scores Permet de renseigner/modifier les scores lorsqu'un match est en cours ou lorsque cette fonction est autorisée. Définir la vente Permet d'activer/configurer la possibilité de vendre le coupon. Partager le coupon Génère un moyen de partager ou d'importer le coupon sur un autre téléphone, notamment via un code. Supprimer de l'historique Supprime le coupon de l'historique local après confirmation. Principe UX Les actions doivent être contextuelles : une action qui n'est pas applicable au statut actuel du coupon doit être masquée ou désactivée. — 5. Page « Détails du pari » En sélectionnant Voir le détail, l'utilisateur accède à une page dédiée. Header • Retour • Détails du pari • Notification • Résumé du coupon On retrouve : • Date et heure • Type de coupon • Numéro • Nombre d'événements • Événements terminés • Cote totale • Mise • Gains potentiels • Statut — 6. Détail des événements Chaque événement du coupon est présenté dans une card. Contenu • Sport • Championnat • Date et heure • Équipe 1 • VS • Équipe 2 • Pronostic • Cote • Statut Exemple : Football · Vietnam · V-League 03.09.2026 — 20:00 Albany Rush VS Aksu Pavlodar Score exact : 5-4 Cote : 69 Statut : Accepté Cela permet à l'utilisateur de comprendre rapidement pourquoi le coupon a telle cote et quel est l'état de chaque événement. — 7. Actions depuis le détail En bas de la page de détail, l'action principale actuelle est : DUPLIQUER LE COUPON DE PARI Cette fonction permet de reprendre rapidement les mêmes sélections pour créer un nouveau coupon. Le bouton doit être suffisamment visible mais ne doit pas concurrencer les informations essentielles du coupon. — 8. Suppression du coupon La suppression doit être protégée par une confirmation. Exemple Supprimer ce coupon ? Ce coupon sera retiré de votre historique. Cette action est irréversible. Annuler | Supprimer L'action destructive doit être clairement différenciée des autres actions. — 9. Logique UX globale Le parcours utilisateur peut être résumé ainsi : HISTORIQUE | Filtres | Période | Type | Statistiques | Liste des coupons | Coupon | Notification | Voir le détail | Détails du pari | Événements | Dupliquer | Saisir les scores | Définir la vente | Partager le coupon | Supprimer Objectif UI/UX L'objectif principal de cette partie est de permettre à l'utilisateur de comprendre son historique en quelques secondes, puis

d'effectuer une action en un minimum de clics. La hiérarchie recommandée est donc : Solde → Filtres → Statistiques → Coupon → Statut → Actions ... → Détail. Le système doit également être responsive, cohérent et orienté mobile, avec des cartes suffisamment espacées, des zones tactiles larges et des confirmations pour les actions sensibles. En résumé La partie Historique regroupe donc 4 niveaux fonctionnels : 1. Consulter → liste des coupons et statistiques. 2. Filtrer → période et type d'opération. 3. Gérer → détail, scores, vente, partage, suppression. 4. Rejouer → duplication d'un coupon existant. C'est cette organisation qui donne à l'historique une logique claire de consultation + gestion + action, tout en gardant l'interface simple pour l'utilisateur.

Response: Ces captures permettent d'identifier précisément l'architecture : il s'agit d'un **générateur / simulateur de coupons réalistes multi-formats** reprenant l'ergonomie complète de l'application bookmaker (onglets, cartes billetterie à encoches circulaires, mini-jeux et gestion interne des matchs).

Voici la feuille de route technique pour monter cette application avec **React Native / Expo**.

1. Navigation & Bottom Bar

L'application repose sur une barre de navigation inférieure à 5 onglets avec un bouton central surélevé :

- **Populaire** : Le hub vers les mini-studios et mini-jeux (Apple of Fortune, Crash, Mortal Kombat, TV Poker).
- **Favoris** : Événements ou sélections épinglés.
- **Coupon (Bouton central surélevé bleu)** : Accès direct au panier / créateur de coupon instantané.
- **Historique** : Liste des tickets générés, solde dynamique, filtres et stats.
- **Menu** : Administration complète (gestion du solde, création manuelle de championnats/équipes, saisie des scores, assistant, apparence).

Pour le bouton central incurvé, un `TouchableOpacity` personnalisé avec un léger offset négatif (`marginTop: -25`) et un fond bleu `#2A52BE` reproduit fidèlement ce rendu.

2. Modèle de données centralisé (Zustand)

Pour que la création dans un studio, la mise à jour des scores et le recalcul des gains dans l'Historique restent synchronisés, le store d'état doit gérer cette structure :

```
// types/bet.ts
export type BetStatus = 'Accepté' | 'En cours' | 'Gagné' | 'Perdu' | 'Annulé';

export interface MatchEvent {
  id: string;
  sport: string;
  league: string;
  date: string;
  homeTeam: { name: string; logo: string };
  awayTeam: { name: string; logo: string };
  prediction: string; // ex: "Score exact: 5-4"
  odd: number; // ex: 69.0
  actualScore?: string;
  status: BetStatus;
}

export interface BetSlip {
  id: string; // ex: "85648871152"
  createdAt: string;
  type: 'Simple' | 'Combiné';
  stake: number; // Mise (ex: 500 F)
  totalOdds: number; // Calculée ou forcée manuellement
  potentialPayout: number; // stake * totalOdds
  status: BetStatus;
  events: MatchEvent[];
  isForSale?: boolean; // Flag "Définir la vente"
}
```

3. Les Écrans clés à coder en priorité

A. La Card "Billet" dans l'Historique & Détail

- Remarquez les demi-cercles sur les côtés des cartes (style ticket de cinéma/pari).
- En React Native, cela se crée simplement en plaçant deux pastilles circulaires absolues (`position: 'absolute', backgroundColor: '#f1f5f9'`) à gauche et à droite de la bordure séparant l'en-tête du ticket et son contenu.

B. Le Bottom Sheet d'actions (Image 7)

- Options déclenchées par les trois points ... :
 1. Voir le détail → Route vers l'écran `BetDetailScreen`.
 2. Saisir les scores → Modale permettant de basculer le statut d'un match (Gain/Perte/Score exact) et de recalculer le coupon.
 3. Définir la vente → Toggle pour faire apparaître le bandeau vert/bleu "Vendre le coupon".

4. Partager le coupon → Export d'un code alphanumérique (ex: A8KF2) ou copie dans le presse-papier via expo-sharing.

5. Supprimer de l'historique → Alert.alert de confirmation destructive rouge.

4. Prochaine étape

Par quoi préfères-tu commencer ?

1. **L'arborescence Expo Router & la Bottom Tab Bar personnalisée** (avec le bouton central de coupon flottant).
2. **Le composant UI du Coupon de pari / Billet** (avec les découpes circulaires et la mise en page exacte de la liste et des détails).
3. **Le module Studio / Saisie manuelle** (l'écran pour créer les faux matchs et générer le numéro de ticket).

User prompt: Voilà les captures du studio poker

Response: C'est super clair avec ces captures ! L'écran **Studio TV (TvBet Poker)** a une logique très bien pensée.

Analysons son fonctionnement exact pour pouvoir le coder :

1. Décomposition de l'écran "Studio TV"

1. Header supérieur :

- Flèche retour ←
- Titre : **Studio TV**
- Icône manette de jeu en haut à droite 🎮

2. Bannière d'information :

- Icône TV / Manette
- Titre : **Coupon TVBet Poker**
- Description : *"Ajoute plusieurs manches. Leurs horaires sont espacés automatiquement de 3 minutes."*

3. Card Programmation :

- Sélecteur de date/heure : **Première manche** (ex: 03.09.2026 (13:32)).
- Note textuelle : *"Les manches suivantes démarrent automatiquement toutes les 3 minutes."*

4. Section Événements (Multi-manches) :

- En-tête : Événement avec compteur (1 manche, 2 manches, etc.) + Bouton bleu + **Ajouter** (qui incrémente l'heure de +3 min pour la manche suivante).
- **Badge numéro de manche** (cercle bleu avec le chiffre 1, 2...) + Libellé TvBet . POKER + Heure calculée.
- **Sélecteur d'onglets Marché (Tabs) :**
 - Combinaison
 - Quelle Main va...
- **Grille de Pronostics (Boutons sélectionnables avec cotes) :**
 - **Onglet Combinaison :**
 - Carte haute : 100
 - Une Paire : 5.
 - Deux Paires : 3.
 - Breton : 6.
 - Quinte : 5.
 - Couleur : 8.
 - Full House : 8.
 - Carré : 10
 - Quinte flush : 60
 - Quinte flush royale : 100
 - Combinaison gagnante. Carte haute : 1000

▪ Onglet Quelle Main va :

- Main 1 : 5.
- Main 2 : 5.
- Main 3 : 5.
- Main 4 : 5.
- Main 5 : 5.
- Main 6 : 5.
- Quelle Main va gagner ?. Main 1 : 5.

5. Section Mise et résumé (En bas) :

- Champ de saisie : **Mise** (ex: 90)
- Checkbox/Option : Mise
- **Cote totale calculée** (ex: 5.)
- **Gains potentiels** (en vert gras, ex: 468 F — calculé selon la mise et la cote avec marge ou bonus)
- Note en gris : *"Les cotes TVBet sont prédéfinies et ne peuvent pas être modifiées."*
- Bouton vert imposant : **Créer le coupon** (ajoute directement le coupon dans l'Historique).

2. Implémentation React Native complète

Voici le code du composant StudioTVScreen.tsx prêt à l'emploi avec StyleSheet standard (compatible Expo sans dépendances complexes) :

```
import React, { useState } from 'react';
import {
  View,
  Text,
  StyleSheet,
  ScrollView,
  TouchableOpacity,
  TextInput,
  SafeAreaView,
  Alert,
} from 'react-native';
import { Ionicons, MaterialCommunityIcons } from '@expo/vector-icons';

interface RoundEvent {
  id: string;
  roundNumber: number;
  timeString: string;
  marketType: 'combination' | 'hand';
  selectedOption: { name: string; odd: number } | null;
}

const COMBINATIONS = [
  { name: 'Carte haute', odd: 100 },
  { name: 'Une Paire', odd: 5 },
  { name: 'Deux Paires', odd: 3 },
  { name: 'Brelan', odd: 6 },
  { name: 'Quinte', odd: 5 },
  { name: 'Couleur', odd: 8 },
  { name: 'Full House', odd: 8 },
  { name: 'Carré', odd: 10 },
  { name: 'Quinte flush', odd: 60 },
  { name: 'Quinte flush royale', odd: 100 },
  { name: 'Combinaison gagnante. Carte haute', odd: 1000 },
];

const HANDS = [
  { name: 'Main 1', odd: 5 },
  { name: 'Main 2', odd: 5 },
  { name: 'Main 3', odd: 5 },
  { name: 'Main 4', odd: 5 },
  { name: 'Main 5', odd: 5 },
  { name: 'Main 6', odd: 5 },
  { name: 'Quelle Main va gagner ?. Main 1', odd: 5 },
];

export default function StudioTVScreen({ navigation }: any) {
  const [baseDate, setBaseDate] = useState('03.09.2026 (13:32)');
  const [stake, setStake] = useState('90');
```

```

// Liste des manches (supporte l'ajout multi-manches)
const [events, setEvents] = useState<RoundEvent[]>([
  {
    id: '1',
    roundNumber: 1,
    timeString: '03.09.2026 (13:32)',
    marketType: 'combination',
    selectedOption: { name: 'Carte haute', odd: 100 },
  },
]);

// Calcul dynamique de la cote totale cumulée
const totalOdd = events.reduce((acc, ev) => {
  return ev.selectedOption ? acc * (ev.selectedOption.odd || 1) : acc;
}, 1);

const numericStake = parseFloat(stake) || 0;
const potentialPayout = Math.round(numericStake * (totalOdd === 1 ? 5.2 : totalOdd));

const addRound = () => {
  const nextRound = events.length + 1;
  // Incrémente l'horaire de 3 minutes pour les suivantes
  const newEvent: RoundEvent = {
    id: String(Date.now()),
    roundNumber: nextRound,
    timeString: `03.09.2026 (13:${32 + (nextRound - 1) * 3})`,
    marketType: 'combination',
    selectedOption: null,
  };
  setEvents([...events, newEvent]);
};

const selectMarket = (eventId: string, type: 'combination' | 'hand') => {
  setEvents(
    events.map((e) =>
      e.id === eventId ? { ...e, marketType: type, selectedOption: null } : e
    )
  );
};

const selectBet = (eventId: string, option: { name: string; odd: number }) => {
  setEvents(
    events.map((e) => (e.id === eventId ? { ...e, selectedOption: option } : e))
  );
};

const handleCreateCoupon = () => {
  Alert.alert(
    'Coupon créé !',
    `Mise: ${numericStake} F\nGains potentiels: ${potentialPayout} F\nLe coupon a été ajouté à votre Historique.`
  );
};

return (
  <SafeAreaView style={styles.container}>
    {/* Header */}
    <View style={styles.header}>
      <TouchableOpacity onPress={() => navigation.goBack()} style={styles.iconBtn}>
        <Ionicons name="arrow-back" size={22} color="#1e293b" />
      </TouchableOpacity>
      <Text style={styles.headerTitle}>Studio TV</Text>
      <TouchableOpacity style={styles.iconBtn}>
        <Ionicons name="game-controller-outline" size={24} color="#94a3b8" />
      </TouchableOpacity>
    </View>

    <ScrollView contentContainerStyle={styles.scrollContent} showsVerticalScrollIndicator={false}>
      {/* Banner Info */}
      <View style={styles.infoBanner}>
        <View style={styles.infoIconBox}>
          <MaterialCommunityIcons name="television-play" size={24} color="#2563eb" />
        </View>
        <View style={{ flex: 1 }}>
          <Text style={styles.infoTitle}>Coupon TVBet Poker</Text>
          <Text style={styles.infoSubtitle}>
            Ajoute plusieurs manches. Leurs horaires sont espacés automatiquement de 3 minutes.
          </Text>
        </View>
      </View>
    </ScrollView>
  </SafeAreaView>
);

```

```

</View>
{ /* Programmation */
<Text style={styles.sectionHeader}>Programmation</Text>
<TouchableOpacity style={styles.dateSelector}>
  <View style={styles.dateIconBox}>
    <Ionicons name="calendar-outline" size={18} color="#2563eb" />
  </View>
  <View style={{ flex: 1 }}>
    <Text style={styles.dateSub}>Première manche</Text>
    <Text style={styles.dateValue}>{baseDate}</Text>
  </View>
  <Ionicons name="chevron-forward" size={18} color="#94a3b8" />
</TouchableOpacity>
<Text style={styles.hintText}>
  Les manches suivantes démarrent automatiquement toutes les 3 minutes.
</Text>

{ /* Événements Header */
<View style={styles.eventsHeader}>
  <View>
    <Text style={styles.sectionHeader}>Événement</Text>
    <Text style={styles.roundCounter}>{events.length} manche{events.length > 1 ? 's' : ''}</Text>
  </View>
  <TouchableOpacity style={styles.addBtn} onPress={addRound}>
    <Ionicons name="add" size={16} color="#fff" />
    <Text style={styles.addBtnText}>Ajouter</Text>
  </TouchableOpacity>
</View>

{ /* Liste des Manches */
events.map((ev) => (
  <View key={ev.id} style={styles.eventCard}>
    { /* Round Title */
    <View style={styles.eventCardHeader}>
      <View style={styles.roundBadge}>
        <Text style={styles.roundBadgeText}>{ev.roundNumber}</Text>
      </View>
      <View>
        <Text style={styles.matchTitle}>TvBet. POKER</Text>
        <Text style={styles.matchTime}>{ev.timeString}</Text>
      </View>
    </View>

    { /* Onglets Choix du Marché */
    <Text style={styles.marketLabel}>Marché</Text>
    <View style={styles.marketTabs}>
      <TouchableOpacity
        style={[
          styles.tabButton,
          ev.marketType === 'combination' && styles.tabButtonActive,
        ]}
        onPress={() => selectMarket(ev.id, 'combination')}
      >
        <Text
          style={[
            styles.tabButtonText,
            ev.marketType === 'combination' && styles.tabButtonTextActive,
          ]}
        >
          Combinaison
        </Text>
      </TouchableOpacity>

      <TouchableOpacity
        style={[
          styles.tabButton,
          ev.marketType === 'hand' && styles.tabButtonActive,
        ]}
        onPress={() => selectMarket(ev.id, 'hand')}
      >
        <Text
          style={[
            styles.tabButtonText,
            ev.marketType === 'hand' && styles.tabButtonTextActive,
          ]}
        >
          Quelle Main va
        </Text>
      </TouchableOpacity>
    }
  )
)
}

```

```

    </TouchableOpacity>
  </View>

  { /* Pronostic List */ }
  <Text style={styles.marketLabel}>Pronostic</Text>
  <View style={styles.oddsGrid}>
    {(ev.marketType === 'combination' ? COMBINATIONS : HANDS).map((item, idx) => {
      const isSelected = ev.selectedOption?.name === item.name;
      const isFullWidth = item.name.length > 20;

      return (
        <TouchableOpacity
          key={idx}
          style={[
            styles.oddBox,
            isFullWidth && styles.oddBoxFull,
            isSelected && styles.oddBoxActive,
          ]}
          onPress={() => selectBet(ev.id, item)}
        >
          <Text
            numberOfLines={1}
            style={[styles.oddName, isSelected && styles.oddTextActive]}
          >
            {item.name}
          </Text>
          <Text
            style={[styles.oddValue, isSelected && styles.oddTextActive]}
          >
            {item.odd}.
          </Text>
        </TouchableOpacity>
      );
    })}
  </View>
</View>
)))

{ /* Mise et résumé */ }
<View style={styles.summaryCard}>
  <Text style={styles.summaryTitle}>Mise et résumé</Text>
  <Text style={styles.inputLabel}>Mise</Text>
  <TextInput
    style={styles.input}
    value={stake}
    onChangeText={setStake}
    keyboardType="numeric"
    placeholder="Montant"
  />

  <View style={styles.summaryRow}>
    <Text style={styles.summaryRowLabel}>Cote</Text>
    <Text style={styles.summaryRowVal}>{totalOdd === 1 ? '5.' : ` ${totalOdd}.`}</Text>
  </View>

  <View style={styles.summaryRow}>
    <Text style={styles.summaryRowLabel}>Gains potentiels</Text>
    <Text style={styles.summaryPayout}>{potentialPayout.toLocaleString()} F</Text>
  </View>

  <Text style={styles.disclaimerText}>
    Les cotes TVBet sont prédéfinies et ne peuvent pas être modifiées.
  </Text>

  <TouchableOpacity style={styles.submitButton} onPress={handleCreateCoupon}>
    <Text style={styles.submitButtonText}>Créer le coupon</Text>
  </TouchableOpacity>
</View>
</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#F3F5F8',
  },
  header: {

```

```
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: 16,
    paddingVertical: 12,
    backgroundColor: '#fff',
    borderBottomWidth: 1,
    borderBottomColor: '#E2E8F0',
  },
  headerTitle: {
    fontSize: 18,
    fontWeight: '700',
    color: '#1E293B',
  },
  iconBtn: {
    padding: 6,
  },
  scrollContent: {
    padding: 16,
    paddingBottom: 40,
  },
  infoBanner: {
    flexDirection: 'row',
    backgroundColor: '#fff',
    borderRadius: 14,
    padding: 14,
    alignItems: 'center',
    marginBottom: 16,
    borderWidth: 1,
    borderColor: '#E2E8F0',
  },
  infoIconBox: {
    width: 44,
    height: 44,
    borderRadius: 10,
    backgroundColor: '#FFF6FF',
    justifyContent: 'center',
    alignItems: 'center',
    marginRight: 12,
  },
  infoTitle: {
    fontSize: 15,
    fontWeight: '700',
    color: '#1E293B',
  },
  infoSubtitle: {
    fontSize: 12,
    color: '#64748B',
    marginTop: 2,
  },
  sectionHeader: {
    fontSize: 15,
    fontWeight: '700',
    color: '#1E293B',
  },
  dateSelector: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#fff',
    borderRadius: 12,
    padding: 12,
    marginTop: 8,
    borderWidth: 1,
    borderColor: '#E2E8F0',
  },
  dateIconBox: {
    width: 36,
    height: 36,
    borderRadius: 8,
    backgroundColor: '#FFF6FF',
    justifyContent: 'center',
    alignItems: 'center',
    marginRight: 12,
  },
  dateSub: {
    fontSize: 12,
    color: '#64748B',
  },
  dateValue: {
```

```
    fontSize: 14,
    fontWeight: '700',
    color: '#1E293B',
    marginTop: 1,
  },
  hintText: {
    fontSize: 11,
    color: '#64748B',
    marginTop: 6,
    marginBottom: 16,
  },
  eventsHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 10,
  },
  roundCounter: {
    fontSize: 12,
    color: '#64748B',
  },
  addBtn: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#1E40AF',
    paddingVertical: 6,
    paddingHorizontal: 12,
    borderRadius: 8,
  },
  addBtnText: {
    color: 'fff',
    fontSize: 13,
    fontWeight: '600',
    marginLeft: 4,
  },
  eventCard: {
    backgroundColor: 'fff',
    borderRadius: 14,
    padding: 14,
    marginBottom: 16,
    borderWidth: 1,
    borderColor: '#E2E8F0',
  },
  eventCardHeader: {
    flexDirection: 'row',
    alignItems: 'center',
    marginBottom: 12,
  },
  roundBadge: {
    width: 28,
    height: 28,
    borderRadius: 14,
    backgroundColor: '#1E40AF',
    justifyContent: 'center',
    alignItems: 'center',
    marginRight: 10,
  },
  roundBadgeText: {
    color: 'fff',
    fontWeight: '700',
    fontSize: 13,
  },
  matchTitle: {
    fontSize: 14,
    fontWeight: '700',
    color: '#1E293B',
  },
  matchTime: {
    fontSize: 11,
    color: '#64748B',
  },
  marketLabel: {
    fontSize: 12,
    fontWeight: '600',
    color: '#475569',
    marginBottom: 8,
    marginTop: 4,
  },
  marketTabs: {
```

```

    flexDirection: 'row',
    gap: 8,
    marginBottom: 12,
  },
  tabButton: {
    flex: 1,
    backgroundColor: '#F1F5F9',
    paddingVertical: 10,
    borderRadius: 8,
    alignItems: 'center',
  },
  tabButtonActive: {
    backgroundColor: '#1E40AF',
  },
  tabButtonText: {
    fontSize: 13,
    color: '#475569',
    fontWeight: '600',
  },
  tabButtonTextActive: {
    color: 'fff',
  },
  oddsGrid: {
    flexDirection: 'row',
    flexWrap: 'wrap',
    gap: 8,
  },
  oddBox: {
    width: '48.5%',
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    backgroundColor: '#F8FAFC',
    borderWidth: 1,
    borderColor: '#E2E8F0',
    borderRadius: 8,
    paddingVertical: 10,
    paddingHorizontal: 12,
  },
  oddBoxFull: {
    width: '100%',
  },
  oddBoxActive: {
    backgroundColor: '#1E40AF',
    borderColor: '#1E40AF',
  },
  oddName: {
    fontSize: 13,
    color: '#1E293B',
    fontWeight: '500',
    flex: 1,
  },
  oddValue: {
    fontSize: 13,
    fontWeight: '700',
    color: '#1E293B',
  },
  oddTextActive: {
    color: 'fff',
  },
  summaryCard: {
    backgroundColor: 'fff',
    borderRadius: 14,
    padding: 16,
    borderWidth: 1,
    borderColor: '#E2E8F0',
  },
  summaryTitle: {
    fontSize: 16,
    fontWeight: '700',
    color: '#1E293B',
    marginBottom: 12,
  },
  inputLabel: {
    fontSize: 12,
    fontWeight: '600',
    color: '#64748B',
    marginBottom: 6,
  },

```

```

input: {
  backgroundColor: '#F1F5F9',
  borderRadius: 10,
  paddingHorizontal: 14,
  paddingVertical: 10,
  fontSize: 16,
  fontWeight: '700',
  color: '#1E293B',
  marginBottom: 14,
},
summaryRow: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  marginBottom: 8,
},
summaryRowLabel: {
  fontSize: 13,
  color: '#64748B',
},
summaryRowVal: {
  fontSize: 14,
  fontWeight: '700',
  color: '#1E293B',
},
summaryPayout: {
  fontSize: 16,
  fontWeight: '800',
  color: '#16A34A',
},
disclaimerText: {
  fontSize: 11,
  color: '#94A3B8',
  marginVertical: 12,
},
submitButton: {
  backgroundColor: '#22C55E',
  borderRadius: 10,
  paddingVertical: 14,
  alignItems: 'center',
},
submitButtonText: {
  color: '#fff',
  fontSize: 15,
  fontWeight: '700',
},
});

```

On continue sur quel écran ensuite ?

- **Studio Mortal Kombat** (pour voir son type de grilles et de pronostics) ?
- Ou l'écran **Historique / Bottom Sheet d'actions** ?

User prompt: Voilà comment se présente le détail de pari du coupon poker dans histoire.

Response: Voici les spécificités visuelles et logiques de cette vue **Détails du pari (Coupon Poker)** par rapport aux matchs de foot classiques :

- **Format du titre de manche** : Icône manette de jeu bleue → TvBet. POKER avec date/heure en dessous (ex. 03.09.2026 (15:03) et 15:06, espacés exactement de 3 minutes).
- **Sous-titre centré** : Le badge textuel grisé POKE.
- **Détail du marché & cote** : Pronostic à gauche (Combinaison gagnante. Carte haute), cote à droite (1000), statut en dessous (Accepté).
- **Champ spécifique TVBet** : L'encart Informations supplémentaires : PB840645 (un identifiant unique de manche généré avec le préfixe PB + 6 chiffres).
- **Découpes "Ticket"** : Des encoches demi-cercles blanches/grises au niveau des séparations entre chaque bloc.

Composant React Native : PokerBetDetailScreen.tsx

Ce composant reproduit fidèlement la mise en page, les calculs et le bouton d'action inférieur.

```

import React from 'react';
import {
  View,
  Text,

```

```

StyleSheet,
ScrollView,
TouchableOpacity,
SafeAreaView,
Alert,
} from 'react-native';
import { Ionicons, MaterialCommunityIcons } from '@expo/vector-icons';

interface PokerEvent {
  id: string;
  gameTitle: string;
  date: string;
  badge: string;
  prediction: string;
  odd: number;
  status: 'Accepté' | 'Gagné' | 'Perdu';
  roundCode: string; // Ex: PB840645
}

interface PokerBetTicket {
  betNumber: string;
  createdAt: string;
  type: 'Simple' | 'Combiné';
  eventsCount: number;
  completedCount: number;
  odds: number;
  stake: number;
  potentialPayout: number;
  status: 'Accepté' | 'Gagné' | 'Perdu';
  events: PokerEvent[];
}

export default function PokerBetDetailScreen({ navigation }: any) {
  // Données types basées sur votre capture
  const betData: PokerBetTicket = {
    betNumber: '85517683766',
    createdAt: '03.09.2026 (15:01)',
    type: 'Combiné',
    eventsCount: 2,
    completedCount: 0,
    odds: 1000000,
    stake: 90,
    potentialPayout: 90000000,
    status: 'Accepté',
    events: [
      {
        id: '1',
        gameTitle: 'TvBet. POKER',
        date: '03.09.2026 (15:03)',
        badge: 'POKE',
        prediction: 'Combinaison gagnante. Carte haute',
        odd: 1000,
        status: 'Accepté',
        roundCode: 'PB840645',
      },
      {
        id: '2',
        gameTitle: 'TvBet. POKER',
        date: '03.09.2026 (15:06)',
        badge: 'POKE',
        prediction: 'Combinaison gagnante. Carte haute',
        odd: 1000,
        status: 'Accepté',
        roundCode: 'PB653039',
      },
    ],
  };

  const handleDuplicate = () => {
    Alert.alert('Coupon Dupliqué', 'Les sélections ont été ajoutées à votre coupon en cours.');
```

```

<Text style={styles.headerTitle}>Détails du</Text>
<View style={styles.headerRight}>
  <TouchableOpacity style={styles.headerBtn}>
    <Ionicons name="notifications-outline" size={22} color="#1E293B" />
  </TouchableOpacity>
  <TouchableOpacity style={styles.headerBtn}>
    <Ionicons name="ellipsis-vertical" size={22} color="#1E293B" />
  </TouchableOpacity>
</View>
</View>

<ScrollView contentContainerStyle={styles.scrollArea} showsVerticalScrollIndicator={false}>
  { /* Ticket Card Container */ }
  <View style={styles.ticketCard}>
    { /* Bloc Entête / Résumé */ }
    <View style={styles.summarySection}>
      <Text style={styles.createdDate}>{betData.createdAt}</Text>
      <Text style={styles.betTitle}>
        {betData.type} <Text style={styles.betNumber}>N° {betData.betNumber}</Text>
      </Text>

      <View style={styles.divider} />

      <View style={styles.rowBetween}>
        <View style={styles.iconLabel}>
          <Ionicons name="receipt-outline" size={18} color="#2563EB" />
          <Text style={styles.labelText}>Nombre d'événements : {betData.eventsCount}</Text>
        </View>
        <Text style={styles.subInfoText}>{betData.completedCount} de {betData.eventsCount} terminés</Text>
      </View>

      <View style={styles.rowBetween}>
        <Text style={styles.labelText}>Cote :</Text>
        <Text style={styles.boldValue}>{betData.odds.toLocaleString()}</Text>
      </View>

      <View style={styles.rowBetween}>
        <Text style={styles.labelText}>Mise :</Text>
        <Text style={styles.boldValue}>{betData.stake} F</Text>
      </View>

      <View style={styles.rowBetween}>
        <Text style={styles.labelText}>Gains potentiels :</Text>
        <Text style={styles.boldValue}>{betData.potentialPayout.toLocaleString()} F</Text>
      </View>

      <View style={styles.rowBetween}>
        <Text style={styles.labelText}>Statut :</Text>
        <View style={styles.statusBadge}>
          <Ionicons name="checkmark-circle" size={16} color="#2563EB" />
          <Text style={styles.statusText}>{betData.status}</Text>
        </View>
      </View>
    </View>

    { /* Découpe perforée (Encoches demi-cercle) */ }
    <View style={styles.cutoutContainer}>
      <View style={[styles.notch, styles.notchLeft]} />
      <View style={styles.dashedSeparator} />
      <View style={[styles.notch, styles.notchRight]} />
    </View>

    { /* Liste des Événements / Manches */ }
    {betData.events.map((event, index) => (
      <React.Fragment key={event.id}>
        <View style={styles.eventSection}>
          { /* Header Événement */ }
          <View style={styles.eventHeader}>
            <MaterialCommunityIcons name="gamepad-variant-outline" size={22} color="#2563EB" />
            <View style={styles.eventTitles}>
              <Text style={styles.eventGame}>{event.gameTitle}</Text>
              <Text style={styles.eventDate}>{event.date}</Text>
            </View>
          </View>

          { /* Badge centré POKE */ }
          <Text style={styles.gamePill}>{event.badge}</Text>

          { /* Pronostic & Cote */ }

```

```

    <View style={styles.rowBetween}>
      <Text style={styles.predictionText}>{event.prediction}</Text>
      <Text style={styles.oddText}>{event.odd}</Text>
    </View>

    {/* Statut Événement */}
    <View style={styles.rowBetween}>
      <Text style={styles.eventLabel}>Statut :</Text>
      <Text style={styles.eventStatusBlue}>{event.status}</Text>
    </View>

    {/* Informations supplémentaires */}
    <View style={styles.infoSuppBlock}>
      <Text style={styles.infoSuppLabel}>Informations supplémentaires :</Text>
      <Text style={styles.roundCode}>{event.roundCode}</Text>
    </View>
  </View>

  {/* Séparateur entre les manches */}
  {index < betData.events.length - 1 && (
    <View style={styles.cutoutContainer}>
      <View style={[styles.notch, styles.notchLeft]} />
      <View style={styles.dashedSeparator} />
      <View style={[styles.notch, styles.notchRight]} />
    </View>
  )}
</React.Fragment>
  )}
</View>

  {/* Bouton Dupliquer le coupon */}
  <TouchableOpacity style={styles duplicateButton} onPress={handleDuplicate}>
    <Text style={styles duplicateButtonText}>DUPLIQUER LE COUPON DE PARI</Text>
  </TouchableOpacity>
</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#F1F5F9',
  },
  header: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: 16,
    paddingVertical: 12,
    backgroundColor: 'fff',
    borderBottomWidth: 1,
    borderBottomColor: '#E2E8F0',
  },
  headerBtn: {
    padding: 6,
  },
  headerTitle: {
    fontSize: 18,
    fontWeight: '700',
    color: '#1E293B',
  },
  headerRight: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 8,
  },
  scrollArea: {
    padding: 16,
    paddingBottom: 40,
  },
  ticketCard: {
    backgroundColor: 'fff',
    borderRadius: 16,
    overflow: 'hidden',
    borderWidth: 1,
    borderColor: '#E2E8F0',
  },
  summarySection: {

```

```
padding: 16,
},
createdDate: {
  fontSize: 13,
  color: '#64748B',
},
betTitle: {
  fontSize: 17,
  fontWeight: '700',
  color: '#1E293B',
  marginTop: 4,
},
betNumber: {
  fontWeight: '600',
},
divider: {
  height: 1,
  backgroundColor: '#F1F5F9',
  marginVertical: 12,
},
rowBetween: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  marginVertical: 4,
},
iconLabel: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 6,
},
labelText: {
  fontSize: 14,
  color: '#64748B',
},
subInfoText: {
  fontSize: 13,
  color: '#64748B',
},
boldValue: {
  fontSize: 15,
  fontWeight: '700',
  color: '#1E293B',
},
statusBadge: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 5,
},
statusText: {
  fontSize: 14,
  fontWeight: '700',
  color: '#2563EB',
},
cutoutContainer: {
  flexDirection: 'row',
  alignItems: 'center',
  height: 20,
  backgroundColor: '#fff',
  position: 'relative',
},
notch: {
  width: 18,
  height: 18,
  borderRadius: 9,
  backgroundColor: '#F1F5F9',
  position: 'absolute',
},
notchLeft: {
  left: -9,
},
notchRight: {
  right: -9,
},
dashedSeparator: {
  flex: 1,
  height: 1,
  borderWidth: 1,
  borderColor: '#E2E8F0',
}
```

```

        borderStyle: 'dashed',
        marginHorizontal: 12,
    },
    eventSection: {
        padding: 16,
    },
    eventHeader: {
        flexDirection: 'row',
        alignItems: 'center',
        gap: 8,
    },
    eventTitles: {
        flex: 1,
    },
    eventGame: {
        fontSize: 14,
        fontWeight: '700',
        color: '#1E293B',
    },
    eventDate: {
        fontSize: 12,
        color: '#64748B',
    },
    gamePill: {
        textAlign: 'center',
        fontSize: 12,
        fontWeight: '700',
        color: '#475569',
        letterSpacing: 1,
        marginVertical: 10,
    },
    predictionText: {
        fontSize: 14,
        fontWeight: '600',
        color: '#1E293B',
        flex: 1,
        paddingRight: 8,
    },
    oddText: {
        fontSize: 15,
        fontWeight: '700',
        color: '#1E293B',
    },
    eventLabel: {
        fontSize: 13,
        color: '#64748B',
    },
    eventStatusBlue: {
        fontSize: 14,
        fontWeight: '600',
        color: '#2563EB',
    },
    infoSuppBlock: {
        marginTop: 10,
    },
    infoSuppLabel: {
        fontSize: 13,
        color: '#64748B',
    },
    roundCode: {
        fontSize: 14,
        fontWeight: '700',
        color: '#1E293B',
        marginTop: 2,
    },
    duplicateButton: {
        backgroundColor: '#DBEAFE',
        borderRadius: 12,
        paddingVertical: 16,
        alignItems: 'center',
        marginTop: 18,
    },
    duplicateButtonText: {
        color: '#2563EB',
        fontSize: 14,
        fontWeight: '800',
        letterSpacing: 0.5,
    },
});

```

Pour enchaîner, veux-tu qu'on prépare l'écran **Studio Mortal Kombat**, ou qu'on branche le passage de paramètres dynamiques (lorsqu'on clique sur "Voir le détail" depuis l'Histoire) ?

User prompt: Voilà comment se présente la validation du coupon poker Et icône tu Tvbet POKER

Response: Cette capture révèle tous les changements visuels et textuels dès qu'un coupon passe de l'état **Accepté** à l'état **Validé / Gagné** :

1. Changements sur le Résumé global

- **Terminés** : 2 de 2 terminés (au lieu de 0 de 2 terminés).
- **Libellé de gain** : Gains : (au lieu de Gains potentiels :).
- **Statut du ticket** :  Payé (avec pastille verte, au lieu du bleu Accepté).

2. Le bloc de tirage centralisé (Cartes & Table) Entre le badge POKE et la sélection de pari, l'affichage détaillé de la manche apparaît en texte centré gris foncé :

- **Main 1** : A♠, J♥
- **Main 2** : 6♥, A♦
- **Main 3** : 6♠, 10♣
- **Main 4** : 3♠, 4♣
- **Main 5** : J♦, 10♣
- **Main 6** : Q♠, 4♦
- **Table** : 2♣, 5♦, 7♠, 9♥, K♣
- **Gagnant** : Gagnant : 1 (Carte haute)

3. Statut de l'événement

- **Statut** :  Gain (en vert avec icône check verte, au lieu du bleu Accepté).

Composant de tirage Poker (React Native)

Voici le composant à insérer dans le ticket pour afficher ce récapitulatif de cartes dès que le statut est validé :

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

interface PokerHandResult {
  hands: { id: number; cards: string }[];
  board: string;
  winnerLabel: string;
}

export const PokerResultBlock = ({ result }: { result: PokerHandResult }) => {
  return (
    <View style={styles.container}>
      <Text style={styles.pokeHeader}>POKE</Text>

      { /* Cartes distribuées pour chaque main */ }
      {result.hands.map((item) => (
        <Text key={item.id} style={styles.handLine}>
          Main {item.id}: {item.cards}
        </Text>
      ))}

      { /* Cartes du Board / Table */ }
      <Text style={[styles.handLine, styles.tableLine]}>
        Table : {result.board}
      </Text>

      { /* Main gagnante */ }
      <Text style={[styles.handLine, styles.winnerLine]}>
        Gagnant : {result.winnerLabel}
      </Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    alignItems: 'center',
```

```

    marginVertical: 8,
  },
  pokeHeader: {
    fontSize: 12,
    fontWeight: '700',
    color: '#334155',
    letterSpacing: 1.2,
    marginBottom: 6,
  },
  handLine: {
    fontSize: 13,
    color: '#334155',
    lineHeight: 18,
    fontWeight: '500',
  },
  tableLine: {
    marginTop: 4,
    fontWeight: '600',
  },
  winnerLine: {
    marginTop: 2,
    fontWeight: '600',
    marginBottom: 8,
  },
  },
});

```

Badge de Statut dynamique (Payé / Gain / Accepté)

Remplace le badge d'état par cette condition :

```

{/* Pour l'en-tête du coupon */}
<View style={styles.statusRow}>
  <Icons
    name="checkmark-circle"
    size={16}
    color={status === 'Payé' || status === 'Gain' ? '#16A34A' : '#2563EB'}
  />
  <Text style={[
    styles.statusText,
    { color: status === 'Payé' || status === 'Gain' ? '#16A34A' : '#2563EB' }
  ]}>
    {status}
  </Text>
</View>

```

Souhaites-tu voir comment générer aléatoirement un tirage de cartes réaliste (avec symboles , , , ) lors de la validation du coupon, ou passe-t-on à un autre studio (Mortal Kombat / Apple of Fortune) ?

User prompt: Oui

Response: Voici l'utilitaire complet pour **simuler et générer un tirage de Poker réaliste** avec les vraies enseignes (, , , ) , attribuer le gagnant en fonction du pari choisi, et mettre à jour le coupon dans l'historique.

1. Le Générateur de Deck & Tirage (pokerEngine.ts)

Ce module s'occupe de mélanger un jeu de 52 cartes, de distribuer les cartes aux 6 mains sans doublon, de sortir le tableau (flop, turn, river) et de formater le résultat exactement comme sur la capture :

```

// utils/pokerEngine.ts

const RANKS = ['2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', 'K', 'A'];
const SUITS = ['

```

```

    for (const s of SUITS) {
      deck.push(`${r}${s}`);
    }
  }

  // 2. Mélange (Fisher-Yates)
  for (let i = deck.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [deck[i], deck[j]] = [deck[j], deck[i]];
  }

  // 3. Distribution de 2 cartes pour 6 mains (12 cartes)
  const hands: HandDistribution[] = [];
  for (let h = 1; h <= 6; h++) {
    const c1 = deck.pop()!;
    const c2 = deck.pop()!;
    hands.push({
      id: h,
      cards: `${c1}, ${c2}`,
    });
  }

  // 4. Distribution de la table (5 cartes : flop, turn, river)
  const boardCards = [
    deck.pop()!,
    deck.pop()!,
    deck.pop()!,
    deck.pop()!,
    deck.pop()!,
  ];
  const board = boardCards.join(' ');

  // 5. Libellé du gagnant
  const winnerLabel = `${winningHandId} (${combinationName})`;

  return {
    hands,
    board,
    winnerLabel,
  };
}

```

2. Action de validation / Saisie des résultats

Dans le store d'état (Zustand ou reducer), voici la fonction à appeler pour passer un coupon de l'état **Accepté** à l'état **Payé** :

```

export const validatePokerCoupon = (couponId: string) => {
  set((state) => ({
    coupons: state.coupons.map((coupon) => {
      if (coupon.id !== couponId) return coupon;

      const updatedEvents = coupon.events.map((event) => {
        // Génère le tirage lié au pronostic sélectionné
        const mockDraw = generateRealisticPokerDraw(1, 'Carte haute');

        return {
          ...event,
          status: 'Gain', // ou 'Perdu'
          pokerResult: mockDraw,
        };
      });

      const allWon = updatedEvents.every((e) => e.status === 'Gain');

      return {
        ...coupon,
        status: allWon ? 'Payé' : 'Perdu',
        completedCount: coupon.eventsCount,
        events: updatedEvents,
      };
    }),
  }));
};

```

3. Affichage conditionnel dans la vue Détails

Dans ton fichier `PokerBetDetailScreen.tsx`, tu bascules l'affichage selon le statut du coupon :

```

{/* Header financier dynamique */}
<View style={styles.rowBetween}>
  <Text style={styles.labelText}>
    {betData.status === 'Payé' ? 'Gains :' : 'Gains potentiels :'}
  </Text>
  <Text style={[[styles.boldValue, betData.status === 'Payé' && { color: '#16A34A' }]]}>
    {betData.potentialPayout.toLocaleString()} F
  </Text>
</View>

<View style={styles.rowBetween}>
  <Text style={styles.labelText}>Statut </Text>
  <View style={styles.statusBadge}>
    <Icons
      name="checkmark-circle"
      size={16}
      color={betData.status === 'Payé' ? '#16A34A' : '#2563EB'}
    />
    <Text style={[[
      styles.statusText,
      { color: betData.status === 'Payé' ? '#16A34A' : '#2563EB' }
    ]]}>
      {betData.status}
    </Text>
  </View>
</View>

{/* Dans chaque événement : affichage du tirage s'il est validé */}
{event.pokerResult ? (
  <PokerResultBlock result={event.pokerResult} />
) : (
  <Text style={styles.gamePill}>{event.badge}</Text>
)}

```

Veux-tu qu'on passe à l'interface du **Studio Mortal Kombat** ou au jeu **Apple of Fortune** ?

User prompt: Passon a l'interface Accéder au studio pour cree tes propres coupons de paris personnalisés

Response: Le **Studio de création de coupons personnalisés** (visible dans l'image 4 et 6 avec le match de football *Albany Rush VS Aksu Pavlodar / Vietnam V-League*) est le cœur de l'application.

Il permet de construire sur-mesure un faux coupon sportif réaliste de A à Z.

1. Analyse UX/UI du Studio Personnalisé

Pour créer un événement de foot comme sur tes captures, l'écran doit proposer :

1. Informations générales du match :

- Choix du sport (Football, Basketball, Tennis, etc.) avec son icône.
- Championnat / Ligue (ex: Football . Vietnam. V-League).
- Date et heure programmées (ex: 03.09.2026 (20:00)).

2. Les deux équipes :

- Équipe 1 (Nom + Logo/Blason).
- Équipe 2 (Nom + Logo/Blason).

3. Le Marché & Pronostic :

- Type de pari (ex: Score exact : 5-4, 1X2, Total Plus/Moins, etc.).
- Cote personnalisée (ex: 69, 55, 1.85).

4. Paramètres financiers du coupon :

- Mise (ex: 500 F).
- Calcul automatique de la cote cumulée ($69 \times 55 = 3\,795$).
- Calcul automatique des gains potentiels ($500 \times 3\,795 = 1\,897\,500$ F).
- Statut initial (Accepté).

2. Implémentation React Native : CustomBetStudioScreen.tsx

Voici le code complet de cet écran générateur de paris :

```

import React, { useState } from 'react';
import {
  View,
  Text,
  StyleSheet,
  ScrollView,
  TextInput,
  TouchableOpacity,
  SafeAreaView,
  Alert,
} from 'react-native';
import { Ionicons, FontAwesome5 } from '@expo/vector-icons';

interface CustomMatch {
  id: string;
  sport: string;
  league: string;
  date: string;
  homeTeam: string;
  awayTeam: string;
  market: string; // Ex: "Score exact : 5-4"
  odd: string;    // Ex: "69"
}

export default function CustomBetStudioScreen({ navigation }: any) {
  const [stake, setStake] = useState('500');

  // Matches configurés dans le coupon
  const [matches, setMatches] = useState<CustomMatch[]>([
    {
      id: '1',
      sport: 'Football',
      league: 'Vietnam. V-League',
      date: '03.09.2026 (20:00)',
      homeTeam: 'Albany Rush',
      awayTeam: 'Aksu Pavlodar',
      market: 'Score exact : 5-4',
      odd: '69',
    },
    {
      id: '2',
      sport: 'Football',
      league: 'Vietnam. V-League',
      date: '03.09.2026 (20:00)',
      homeTeam: 'Aberdeen Fc',
      awayTeam: 'Aarau',
      market: 'Score exact : 4-4',
      odd: '55',
    },
  ]);

  // Calcul dynamique de la cote totale
  const totalOdds = matches.reduce((acc, m) => {
    const val = parseFloat(m.odd) || 1;
    return acc * val;
  }, 1);

  const numericStake = parseFloat(stake) || 0;
  const potentialPayout = Math.round(numericStake * totalOdds);

  const addEmptyMatch = () => {
    const newMatch: CustomMatch = {
      id: String(Date.now()),
      sport: 'Football',
      league: 'Championnat / Tournoi',
      date: '03.09.2026 (20:00)',
      homeTeam: 'Équipe A',
      awayTeam: 'Équipe B',
      market: 'Vainqueur : Équipe A',
      odd: '2.0',
    };
    setMatches([...matches, newMatch]);
  };

  const updateMatchField = (id: string, field: keyof CustomMatch, value: string) => {
    setMatches(matches.map((m) => (m.id === id ? { ...m, [field]: value } : m)));
  };

  const removeMatch = (id: string) => {

```

```

    if (matches.length <= 1) {
      Alert.alert('Action impossible', 'Le coupon doit contenir au moins 1 événement.');
```

return;

```

    }
    setMatches(matches.filter((m) => m.id !== id));
  };

const handleGenerateTicket = () => {
  const ticketNumber = Math.floor(1000000000 + Math.random() * 9000000000).toString();
  Alert.alert(
    'Coupon Enregistré !',
    `Coupon Combiné № ${ticketNumber}\nCote: ${Math.round(totalOdds)}\nGains: ${potentialPayout.toLocaleString()}`
  );
};

return (
  <SafeAreaView style={styles.container}>
    {/* Header */}
    <View style={styles.header}>
      <TouchableOpacity onPress={() => navigation.goBack()} style={styles.iconBtn}>
        <Ionicons name="arrow-back" size={24} color="#1E293B" />
      </TouchableOpacity>
      <Text style={styles.headerTitle}>Studio de création</Text>
      <TouchableOpacity onPress={addEmptyMatch} style={styles.addEventBtn}>
        <Ionicons name="add-circle" size={24} color="#2563EB" />
      </TouchableOpacity>
    </View>

    <ScrollView contentContainerStyle={styles.scrollContent} showsVerticalScrollIndicator={false}>
      {/* En-tête informatif */}
      <View style={styles.infoBox}>
        <Ionicons name="sparkles" size={22} color="#2563EB" />
        <Text style={styles.infoBoxText}>
          Personnalise chaque événement, nom d'équipe, pronostic et cote manuellement.
        </Text>
      </View>

      {/* Liste des cartes d'événements modifiables */}
      {matches.map((item, index) => (
        <View key={item.id} style={styles.matchCard}>
          <View style={styles.cardTopRow}>
            <View style={styles.badgeIndex}>
              <Text style={styles.badgeText}>{index + 1}</Text>
            </View>
            <TextInput
              style={styles.inputLeague}
              value={item.league}
              onChangeText={(text) => updateMatchField(item.id, 'league', text)}
              placeholder="Discipline · Ligue"
            />
            <TouchableOpacity onPress={() => removeMatch(item.id)} style={styles.deleteBtn}>
              <Ionicons name="trash-outline" size={18} color="#EF4444" />
            </TouchableOpacity>
          </View>

          {/* Date & Heure */}
          <TextInput
            style={styles.inputDate}
            value={item.date}
            onChangeText={(text) => updateMatchField(item.id, 'date', text)}
            placeholder="Date (ex: 03.09.2026 (20:00))"
          />

          {/* Équipes : Domicile VS Extérieur */}
          <View style={styles.teamsRow}>
            <TextInput
              style={[[styles.inputTeam, { textAlign: 'left' }]]}
              value={item.homeTeam}
              onChangeText={(text) => updateMatchField(item.id, 'homeTeam', text)}
              placeholder="Équipe Domicile"
            />
            <Text style={styles.vsText}>VS</Text>
            <TextInput
              style={[[styles.inputTeam, { textAlign: 'right' }]]}
              value={item.awayTeam}
              onChangeText={(text) => updateMatchField(item.id, 'awayTeam', text)}
              placeholder="Équipe Extérieur"
            />
          </View>
        </View>
      ))}
    </ScrollView>
  </SafeAreaView>
);

```

```

    { /* Pronostic & Cote */
    <View style={styles.marketRow}>
      <View style={{ flex: 2 }}>
        <Text style={styles.fieldLabel}>Pronostic</Text>
        <TextInput
          style={styles.inputField}
          value={item.market}
          onChangeText={(text) => updateMatchField(item.id, 'market', text)}
          placeholder="Ex: Score exact: 5-4"
        />
      </View>
      <View style={{ flex: 1, marginLeft: 10 }}>
        <Text style={styles.fieldLabel}>Cote</Text>
        <TextInput
          style={[styles.inputField, styles.oddInput]}
          value={item.odd}
          onChangeText={(text) => updateMatchField(item.id, 'odd', text)}
          keyboardType="numeric"
          placeholder="Cote"
        />
      </View>
    </View>
  </View>
))}

{ /* Synthèse financière */
<View style={styles.summaryCard}>
  <Text style={styles.summaryTitle}>Paramètres du coupon</Text>

  <Text style={styles.fieldLabel}>Mise (F)</Text>
  <TextInput
    style={styles.inputStake}
    value={stake}
    onChangeText={setStake}
    keyboardType="numeric"
    placeholder="500"
  />

  <View style={styles.summaryLine}>
    <Text style={styles.summaryLabel}>Nombre de sélections :</Text>
    <Text style={styles.summaryValue}>{matches.length}</Text>
  </View>

  <View style={styles.summaryLine}>
    <Text style={styles.summaryLabel}>Cote globale :</Text>
    <Text style={styles.summaryValue}>{Math.round(totalOdds).toLocaleString()}</Text>
  </View>

  <View style={styles.summaryLine}>
    <Text style={styles.summaryLabel}>Gains potentiels :</Text>
    <Text style={styles.payoutText}>{potentialPayout.toLocaleString()} F</Text>
  </View>

  <TouchableOpacity style={styles.generateBtn} onPress={handleGenerateTicket}>
    <Text style={styles.generateBtnText}>Générer le coupon dans l'Historique</Text>
  </TouchableOpacity>
</View>
</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#F1F5F9',
  },
  header: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: 16,
    paddingVertical: 12,
    backgroundColor: '#fff',
    borderBottomWidth: 1,
    borderBottomColor: '#E2E8F0',
  },
  iconBtn: {

```

```
padding: 6,
},
headerTitle: {
  fontSize: 18,
  fontWeight: '700',
  color: '#1E293B',
},
addEventBtn: {
  padding: 4,
},
scrollContent: {
  padding: 16,
  paddingBottom: 40,
},
infoBox: {
  flexDirection: 'row',
  alignItems: 'center',
  backgroundColor: '#DBEAFE',
  padding: 12,
  borderRadius: 12,
  marginBottom: 16,
  gap: 10,
},
infoBoxText: {
  flex: 1,
  fontSize: 12,
  color: '#1E40AF',
  fontWeight: '500',
},
matchCard: {
  backgroundColor: 'fff',
  borderRadius: 14,
  padding: 14,
  marginBottom: 14,
  borderWidth: 1,
  borderColor: '#E2E8F0',
},
cardTopRow: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 8,
  marginBottom: 8,
},
badgeIndex: {
  width: 24,
  height: 24,
  borderRadius: 12,
  backgroundColor: '#2563EB',
  alignItems: 'center',
  justifyContent: 'center',
},
badgeText: {
  color: 'fff',
  fontSize: 12,
  fontWeight: '700',
},
inputLeague: {
  flex: 1,
  fontSize: 13,
  fontWeight: '600',
  color: '#1E293B',
  paddingVertical: 2,
},
deleteBtn: {
  padding: 4,
},
inputDate: {
  fontSize: 11,
  color: '#64748B',
  marginBottom: 10,
},
teamsRow: {
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-between',
  backgroundColor: '#F8FAFC',
  borderRadius: 8,
  padding: 10,
  marginBottom: 12,
```

```
borderWidth: 1,
borderColor: '#E2E8F0',
},
inputTeam: {
  flex: 1,
  fontSize: 14,
  fontWeight: '700',
  color: '#1E293B',
},
vsText: {
  fontSize: 12,
  fontWeight: '800',
  color: '#94A3B8',
  marginHorizontal: 8,
},
marketRow: {
  flexDirection: 'row',
  alignItems: 'center',
},
fieldLabel: {
  fontSize: 11,
  color: '#64748B',
  marginBottom: 4,
},
inputField: {
  backgroundColor: '#F8FAFC',
  borderWidth: 1,
  borderColor: '#E2E8F0',
  borderRadius: 8,
  paddingHorizontal: 10,
  paddingVertical: 8,
  fontSize: 13,
  color: '#1E293B',
  fontWeight: '600',
},
oddInput: {
  textAlign: 'center',
  fontWeight: '800',
  color: '#2563EB',
},
summaryCard: {
  backgroundColor: '#fff',
  borderRadius: 14,
  padding: 16,
  borderWidth: 1,
  borderColor: '#E2E8F0',
  marginTop: 8,
},
summaryTitle: {
  fontSize: 16,
  fontWeight: '700',
  color: '#1E293B',
  marginBottom: 12,
},
inputStake: {
  backgroundColor: '#F1F5F9',
  borderRadius: 8,
  paddingHorizontal: 12,
  paddingVertical: 10,
  fontSize: 16,
  fontWeight: '700',
  color: '#1E293B',
  marginBottom: 14,
},
summaryLine: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  marginBottom: 8,
},
summaryLabel: {
  fontSize: 13,
  color: '#64748B',
},
summaryValue: {
  fontSize: 14,
  fontWeight: '700',
  color: '#1E293B',
},
payoutText: {
```

```

    fontSize: 16,
    fontWeight: '800',
    color: '#16A34A',
  },
  generateBtn: {
    backgroundColor: '#2563EB',
    borderRadius: 10,
    paddingVertical: 14,
    alignItems: 'center',
    marginTop: 14,
  },
  generateBtnText: {
    color: '#fff',
    fontSize: 15,
    fontWeight: '700',
  },
});

```

Prochain élément

Veux-tu qu'on enchaîne avec :

1. **Le sélecteur de logos d'équipes** (pour choisir les blasons du match comme *Albany Rush* et *Aksu Pavlodar*) ?
2. **Le mini-jeu / Studio Mortal Kombat ?**
3. **Le mini-jeu Apple of Fortune ?**

User prompt: *Voilà les capture du studio de paris*

Response: Ces captures montrent l'implémentation exacte du **Studio de paris personnalisés** (avec tout son formulaire guidé et ses options d'affichage).

Voici les 4 blocs clés visibles sur l'écran :

1. Bannière d'accueil :

- Badge bleu coupon + titre **Création guidée**
- *"ID, équipes, mise et résumé dans un seul parcours."*

2. Section IDENTITÉ :

- Champ de saisie : **ID personnalisé optionnel** (placeholder : Ex: 82574662089)
- Note d'aide : *"11 chiffres, commence par 85. Vide = numéro généré automatiquement. Le code partageable reste séparé."*
- Checkbox : **Ajouter un badge au...**
- Checkbox : ☒ **Badge « En direct » sur le...**

3. Section ÉVÉNEMENT :

- **Source** : Sélecteur horizontal à bascule (Tout, FIFA, Original)
- **Championnat** : Bascule (Base / Custom)
- Sélecteur déroulant / recherche : 🔍 Championnat
- **Marché** : Bascule (Base / Custom) + barre de recherche (*"Rechercher un label, ex: corners, total, carton"*)
- Filtres horizontaux de marché : Résultat, Double chance, Score, Stats...
- Menus d'options: Premier événement, Le Gardien de but touchera le Ballon pendant la Première minute du Match
- Champ **Cote** (ex: 1.85)
- Switch **Score final**
- Bouton **+ Ajouter** (désactivé si le championnat n'est pas choisi : *"Championnat requis"*)

4. Section MISE ET RÉSUMÉ :

- Compteur d'événements créés (ÉVÉNEMENTS (0) → *"Ajoute au moins un événement pour sauvegarder le coupon."*)
- Champ **Mise virtuelle** (ex: 500)
- Checkbox **Mise**
- Checkbox **Afficher le badge dans l'historique** (*"Ajoute un badge personnalisé à au moins un événement pour l'activer."*)
- Checkbox **Activer la vente (cashout)** (*"Affiche un bouton « Vendre pour X F » sur le détail du pari."*)

- Récapitulatif : Événements : 0, Cote : -, Gains : -
- Ligne d'horodatage : Date du pari : Auto (maintenant)

Composant React Native complet : StudioCreationScreen.tsx

```
import React, { useState } from 'react';
import {
  View,
  Text,
  StyleSheet,
  ScrollView,
  TextInput,
  TouchableOpacity,
  Switch,
  SafeAreaView,
  Alert,
} from 'react-native';
import { Ionicons, MaterialCommunityIcons } from '@expo/vector-icons';

export default function StudioCreationScreen({ navigation }: any) {
  // 1. Identité
  const [customId, setCustomId] = useState('');
  const [hasCustomBadge, setHasCustomBadge] = useState(false);
  const [isLiveBadge, setIsLiveBadge] = useState(false);

  // 2. Filtres & Marchés
  const [sourceType, setSourceType] = useState<'Tout' | 'FIFA' | 'Original'>('Tout');
  const [champType, setChampType] = useState<'Base' | 'Custo'>('Base');
  const [selectedLeague, setSelectedLeague] = useState('');
  const [marketType, setMarketType] = useState<'Base' | 'Custo'>('Base');
  const [selectedCategory, setSelectedCategory] = useState('Résultat');
  const [selectedOptionLabel, setSelectedOptionLabel] = useState(
    'Le Gardien de but touchera le Ballon pendant la Première minute du Match'
  );
  const [odd, setOdd] = useState('1.85');
  const [finalScoreSwitch, setFinalScoreSwitch] = useState(false);

  // 3. Événements ajoutés
  const [addedEvents, setAddedEvents] = useState<any[]>([]);

  // 4. Mise & Résumé
  const [stake, setStake] = useState('500');
  const [enableStakeBadge, setEnableStakeBadge] = useState(false);
  const [enableCashout, setEnableCashout] = useState(false);

  // Calculs dynamiques
  const totalOdds = addedEvents.reduce((acc, ev) => acc * (parseFloat(ev.odd) || 1), 1);
  const numericStake = parseFloat(stake) || 0;
  const potentialPayout = addedEvents.length > 0 ? Math.round(numericStake * totalOdds) : 0;

  const handleAddEvent = () => {
    if (!selectedLeague) {
      Alert.alert('Championnat requis', 'Veuillez sélectionner un championnat avant d\'ajouter.');


```

```

<TouchableOpacity onPress={() => navigation?.goBack()} style={styles.backBtn}>
  <Ionicons name="arrow-back" size={24} color="#1E293B" />
</TouchableOpacity>
<Text style={styles.headerTitle}>Studio de création</Text>
<View style={{ width: 24 }} />
</View>

<ScrollView contentContainerStyle={styles.scrollArea} showsVerticalScrollIndicator={false}>
  { /* En-tête : Création guidée */ }
  <View style={styles.bannerBox}>
    <View style={styles.bannerIconBox}>
      <MaterialCommunityIcons name="ticket-percent-outline" size={22} color="#2563EB" />
    </View>
    <View style={{ flex: 1 }}>
      <Text style={styles.bannerTitle}>Création guidée</Text>
      <Text style={styles.bannerSubtitle}>ID, équipes, mise et résumé dans un seul parcours.</Text>
    </View>
  </View>

  { /* Section 1 : IDENTITÉ */ }
  <View style={styles.card}>
    <Text style={styles.cardSectionTitle}>IDENTITÉ</Text>
    <Text style={styles.fieldLabel}>ID personnalisé optionnel</Text>
    <TextInput
      style={styles.input}
      placeholder="Ex: 82574662089"
      value={customId}
      onChangeText={setCustomId}
      keyboardType="numeric"
    />
    <Text style={styles.helperText}>
      11 chiffres, commence par 85. Vide = numéro généré automatiquement. Le code partageable reste séparé.
    </Text>

    { /* Checkbox 1 */ }
    <TouchableOpacity
      style={styles.checkboxRow}
      onPress={() => setHasCustomBadge(!hasCustomBadge)}
    >
      <Ionicons
        name={hasCustomBadge ? 'checkbox' : 'square-outline'}
        size={20}
        color={hasCustomBadge ? '#2563EB' : '#94A3B8'}
      />
      <Text style={styles.checkboxLabel}>Ajouter un badge au coupon</Text>
    </TouchableOpacity>

    { /* Checkbox 2 (Badge En Direct) */ }
    <TouchableOpacity
      style={styles.checkboxRow}
      onPress={() => setIsLiveBadge(!isLiveBadge)}
    >
      <Ionicons
        name={isLiveBadge ? 'checkbox' : 'square-outline'}
        size={20}
        color={isLiveBadge ? '#2563EB' : '#94A3B8'}
      />
      <Text style={styles.checkboxLabel}>
        <Text style={{ color: '#EF4444' }}>• </Text>Badge « En direct » sur le match
      </Text>
    </TouchableOpacity>
  </View>

  { /* Section 2 : ÉVÉNEMENT */ }
  <View style={styles.card}>
    <Text style={styles.cardSectionTitle}>ÉVÉNEMENT</Text>

    { /* Source Filter */ }
    <View style={styles.toggleRow}>
      <Text style={styles.toggleLabel}>Source</Text>
      <View style={styles.pillGroup}>
        {[ 'Tout', 'FIFA', 'Original' ] as const}.map((s) => (
          <TouchableOpacity
            key={s}
            style={[styles.pill, sourceType === s && styles.pillActive]}
            onPress={() => setSourceType(s)}
          >
            <Text style={[styles.pillText, sourceType === s && styles.pillTextActive]}>{s}</Text>
          </TouchableOpacity>
        ))
      </View>
    </View>
  </View>

```

```

    )))
  </View>
</View>

{/* Championnat Filter & Dropdown */}
<View style={styles.toggleRow}>
  <Text style={styles.toggleLabel}>Championnat</Text>
  <View style={styles.pillGroup}>
    {(['Base', 'Custo'] as const).map((c) => (
      <TouchableOpacity
        key={c}
        style={[[styles.pill, champType === c && styles.pillActive]]}
        onPress={() => setChampType(c)}
      >
        <Text style={[[styles.pillText, champType === c && styles.pillTextActive]]}>{c}</Text>
      </TouchableOpacity>
    ))}
  </View>
</View>

<TouchableOpacity
  style={styles.dropdownSelector}
  onPress={() => setSelectedLeague(selectedLeague ? '' : 'Vietnam. V-League')}
>
  <Ionicons name="search-outline" size={16} color="#64748B" />
  <Text style={[[styles.dropdownText, selectedLeague ? { color: '#1E293B', fontWeight: '700' } : null]]}>
    {selectedLeague || 'Championnat'}
  </Text>
  <Ionicons name="chevron-down" size={16} color="#64748B" />
</TouchableOpacity>

{/* Marché */}
<View style={[[styles.toggleRow, { marginTop: 12 }]]}>
  <Text style={styles.toggleLabel}>Marché</Text>
  <View style={styles.pillGroup}>
    {(['Base', 'Custo'] as const).map((m) => (
      <TouchableOpacity
        key={m}
        style={[[styles.pill, marketType === m && styles.pillActive]]}
        onPress={() => setMarketType(m)}
      >
        <Text style={[[styles.pillText, marketType === m && styles.pillTextActive]]}>{m}</Text>
      </TouchableOpacity>
    ))}
  </View>
</View>

<TextInput
  style={[[styles.input, { marginTop: 8 }]]}
  placeholder="Rechercher un label, ex: corners, total, carton"
/>

{/* Marché Categories Tabs */}
<ScrollView horizontal showsHorizontalScrollIndicator={false} style={styles.subTabsContainer}>
  {['Résultat', 'Double chance', 'Score', 'Stats'].map((cat) => (
    <TouchableOpacity
      key={cat}
      style={[[styles.subTab, selectedCategory === cat && styles.subTabActive]]}
      onPress={() => setSelectedCategory(cat)}
    >
      <Text style={[[styles.subTabText, selectedCategory === cat && styles.subTabTextActive]]}>{cat}</Text>
    </TouchableOpacity>
  ))}
</ScrollView>

{/* Sélecteurs de Paris */}
<TouchableOpacity style={styles.menuRowItem}>
  <Text style={styles.menuRowText}>Premier événement</Text>
  <Ionicons name="chevron-forward" size={16} color="#94A3B8" />
</TouchableOpacity>

<TouchableOpacity style={styles.menuRowItem}>
  <Text numberOfLines={2} style={[[styles.menuRowText, { flex: 1 }]]}>
    {selectedOptionLabel}
  </Text>
  <Ionicons name="chevron-forward" size={16} color="#94A3B8" />
</TouchableOpacity>

{/* Cote */}

```

```

<Text style={[[styles.fieldLabel, { marginTop: 10 }]]}>Cote</Text>
<TextInput
  style={[[styles.input, { width: 100, fontWeight: '700' }]]}
  value={odd}
  onChangeText={setOdd}
  keyboardType="numeric"
/>

{!selectedLeague && <Text style={styles.errorHint}>Championnat requis</Text>}

{/* Switch Score final */}
<View style={styles.switchRow}>
  <Text style={styles.switchLabel}>Score final</Text>
  <Switch
    value={finalScoreSwitch}
    onValueChange={setFinalScoreSwitch}
    thumbColor="#fff"
    trackColor={{ false: '#CBD5E1', true: '#2563EB' }}
  />
</View>

{/* Bouton Ajouter */}
<TouchableOpacity
  style={[[styles.addBtn, !selectedLeague && styles.addBtnDisabled]]}
  onPress={handleAddEvent}
>
  <Icons name="add" size={18} color="#fff" />
  <Text style={styles.addBtnText}>Ajouter</Text>
</TouchableOpacity>
</View>

{/* Section 3 : ÉVÉNEMENTS (Liste) */}
<View style={styles.card}>
  <Text style={styles.cardSectionTitle}>ÉVÉNEMENTS ({addedEvents.length})</Text>
  {addedEvents.length === 0 ? (
    <Text style={styles.emptyNote}>Ajoute au moins un événement pour sauvegarder le coupon.</Text>
  ) : (
    addedEvents.map((ev, idx) => (
      <View key={ev.id} style={styles.addedItem}>
        <Text style={styles.addedIndex}>{idx + 1}</Text>
        <View style={{ flex: 1 }}>
          <Text style={styles.addedLeague}>{ev.league}</Text>
          <Text style={styles.addedLabel}>{ev.label}</Text>
        </View>
        <Text style={styles.addedOdd}>{ev.odd}</Text>
      </View>
    ))
  )}
</View>

{/* Section 4 : MISE ET RÉSUMÉ */}
<View style={styles.card}>
  <Text style={styles.cardSectionTitle}>MISE ET RÉSUMÉ</Text>
  <Text style={styles.fieldLabel}>Mise virtuelle</Text>
  <TextInput
    style={[[styles.input, { fontWeight: '700' }]]}
    value={stake}
    onChangeText={setStake}
    keyboardType="numeric"
  />

  {/* Checkbox Mise */}
  <TouchableOpacity
    style={styles.checkboxRow}
    onPress={() => setEnableStakeBadge(!enableStakeBadge)}
  >
    <Icons
      name={enableStakeBadge ? 'checkbox' : 'square-outline'}
      size={20}
      color={enableStakeBadge ? '#2563EB' : '#94A3B8'}
    />
    <Text style={styles.checkboxLabel}>Mise</Text>
  </TouchableOpacity>

  {/* Checkbox Cashout */}
  <TouchableOpacity
    style={styles.checkboxRow}
    onPress={() => setEnableCashout(!enableCashout)}
  >

```

```

<Ionicons
  name={enableCashout ? 'checkbox' : 'square-outline'}
  size={20}
  color={enableCashout ? '#2563EB' : '#94A3B8'}
/>
<View style={{ flex: 1 }}>
  <Text style={styles.checkboxLabel}>Activer la vente (cashout)</Text>
  <Text style={styles.helperText}>
    Affiche un bouton « Vendre pour X F » sur le détail du pari.
  </Text>
</View>
</TouchableOpacity>

{/* Encart Statistiques */}
<View style={styles.statBox}>
  <View style={styles.statLine}>
    <Text style={styles.statLabel}>Événements</Text>
    <Text style={styles.statVal}>{addedEvents.length}</Text>
  </View>
  <View style={styles.statLine}>
    <Text style={styles.statLabel}>Cote</Text>
    <Text style={styles.statVal}>{addedEvents.length > 0 ? totalOdds.toFixed(2) : '-'}</Text>
  </View>
  <View style={styles.statLine}>
    <Text style={styles.statLabel}>Gains</Text>
    <Text style={styles.statVal}>{addedEvents.length > 0 ? `${potentialPayout.toLocaleString()} F` : '-'}</Text>
  </View>
</View>

<Text style={styles.dateAutoText}>Date du pari : Auto (maintenant)</Text>

<TouchableOpacity style={styles.saveBtn} onPress={handleSaveCoupon}>
  <Text style={styles.saveBtnText}>Sauvegarder le coupon</Text>
</TouchableOpacity>
</View>
</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#F1F5F9',
  },
  header: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: 16,
    paddingVertical: 12,
    backgroundColor: 'fff',
    borderBottomWidth: 1,
    borderBottomColor: '#E2E8F0',
  },
  backBtn: {
    padding: 4,
  },
  headerTitle: {
    fontSize: 18,
    fontWeight: '700',
    color: '#1E293B',
  },
  scrollArea: {
    padding: 16,
    paddingBottom: 50,
  },
  bannerBox: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#E0E7FF',
    borderRadius: 14,
    padding: 14,
    marginBottom: 14,
    gap: 12,
  },
  bannerIconBox: {
    width: 38,
    height: 38,
  }
});

```

```
borderRadius: 10,
backgroundColor: '#fff',
justifyContent: 'center',
alignItems: 'center',
},
bannerTitle: {
  fontSize: 15,
  fontWeight: '700',
  color: '#1E293B',
},
bannerSubtitle: {
  fontSize: 12,
  color: '#475569',
  marginTop: 2,
},
card: {
  backgroundColor: '#fff',
  borderRadius: 14,
  padding: 14,
  marginBottom: 14,
  borderWidth: 1,
  borderColor: '#E2E8F0',
},
cardSectionTitle: {
  fontSize: 14,
  fontWeight: '800',
  color: '#1E40AF',
  marginBottom: 10,
},
fieldLabel: {
  fontSize: 12,
  fontWeight: '600',
  color: '#64748B',
  marginBottom: 6,
},
input: {
  backgroundColor: '#F8F AFC',
  borderRadius: 8,
  borderWidth: 1,
  borderColor: '#E2E8F0',
  paddingHorizontal: 12,
  paddingVertical: 10,
  fontSize: 14,
  color: '#1E293B',
},
helperText: {
  fontSize: 11,
  color: '#64748B',
  marginTop: 6,
  lineHeight: 15,
},
checkboxRow: {
  flexDirection: 'row',
  alignItems: 'flex-start',
  gap: 8,
  marginTop: 12,
},
checkboxLabel: {
  fontSize: 13,
  fontWeight: '600',
  color: '#1E293B',
},
toggleRow: {
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-between',
  marginTop: 8,
},
toggleLabel: {
  fontSize: 13,
  color: '#475569',
  fontWeight: '600',
},
pillGroup: {
  flexDirection: 'row',
  backgroundColor: '#F1F5F9',
  borderRadius: 8,
  padding: 2,
},
```

```

pill: {
  paddingVertical: 6,
  paddingHorizontal: 12,
  borderRadius: 6,
},
pillActive: {
  backgroundColor: '#1E40AF',
},
pillText: {
  fontSize: 12,
  fontWeight: '600',
  color: '#64748B',
},
pillTextActive: {
  color: 'fff',
},
dropdownSelector: {
  flexDirection: 'row',
  alignItems: 'center',
  backgroundColor: '#F8FAFC',
  borderWidth: 1,
  borderColor: '#E2E8F0',
  borderRadius: 8,
  paddingHorizontal: 12,
  paddingVertical: 12,
  marginTop: 10,
  gap: 8,
},
dropdownText: {
  flex: 1,
  fontSize: 13,
  color: '#94A3B8',
},
subTabsContainer: {
  flexDirection: 'row',
  marginTop: 10,
  marginBottom: 8,
},
subTab: {
  backgroundColor: '#F1F5F9',
  borderRadius: 8,
  paddingVertical: 6,
  paddingHorizontal: 12,
  marginRight: 6,
},
subTabActive: {
  backgroundColor: '#2563EB',
},
subTabText: {
  fontSize: 12,
  fontWeight: '600',
  color: '#64748B',
},
subTabTextActive: {
  color: 'fff',
},
menuRowItem: {
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-between',
  backgroundColor: '#F8FAFC',
  borderWidth: 1,
  borderColor: '#E2E8F0',
  borderRadius: 8,
  padding: 12,
  marginTop: 8,
},
menuRowText: {
  fontSize: 13,
  fontWeight: '600',
  color: '#1E293B',
},
errorHint: {
  fontSize: 11,
  color: '#EF4444',
  marginTop: 6,
  fontWeight: '600',
},
switchRow: {

```

```

    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginTop: 12,
  },
  switchLabel: {
    fontSize: 13,
    fontWeight: '600',
    color: '#1E293B',
  },
  addBtn: {
    flexDirection: 'row',
    backgroundColor: '#1E40AF',
    borderRadius: 8,
    paddingVertical: 10,
    alignItems: 'center',
    justifyContent: 'center',
    marginTop: 14,
    gap: 6,
  },
  addBtnDisabled: {
    backgroundColor: '#93C5FD',
  },
  addBtnText: {
    color: '#fff',
    fontWeight: '700',
    fontSize: 14,
  },
  emptyNote: {
    fontSize: 12,
    color: '#64748B',
    fontStyle: 'italic',
  },
  addedItem: {
    flexDirection: 'row',
    alignItems: 'center',
    paddingVertical: 6,
    borderBottomWidth: 1,
    borderBottomColor: '#F1F5F9',
  },
  addedIndex: {
    fontSize: 13,
    fontWeight: '700',
    marginRight: 8,
    color: '#2563EB',
  },
  addedLeague: {
    fontSize: 11,
    color: '#64748B',
  },
  addedLabel: {
    fontSize: 13,
    fontWeight: '600',
    color: '#1E293B',
  },
  addedOdd: {
    fontSize: 14,
    fontWeight: '700',
    color: '#2563EB',
  },
  statBox: {
    backgroundColor: '#F8FAFC',
    borderRadius: 8,
    padding: 12,
    marginTop: 14,
    borderWidth: 1,
    borderColor: '#E2E8F0',
  },
  statline: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginVertical: 3,
  },
  statLabel: {
    fontSize: 13,
    color: '#64748B',
  },
  statVal: {
    fontSize: 13,

```

```

fontWeight: '700',
color: '#1E293B',
},
dateAutoText: {
  fontSize: 12,
  color: '#64748B',
  marginVertical: 12,
  textAlign: 'center',
},
saveBtn: {
  backgroundColor: '#22C55E',
  borderRadius: 10,
  paddingVertical: 14,
  alignItems: 'center',
},
saveBtnText: {
  color: 'fff',
  fontSize: 15,
  fontWeight: '700',
},
});

```

Veux-tu qu'on passe aux captures de l'écran **Studio Mortal Kombat** ou à celles du jeu **Apple of Fortune** ?

User prompt: Pour le moment génère moi un fichier zip de ce projet

Response: En tant qu'assistant textuel, je ne peux pas directement vous envoyer un fichier binaire .zip téléchargeable.

En revanche, **voici l'architecture complète prête à l'emploi**. En exécutant les commandes ci-dessous dans votre terminal, vous obtiendrez exactement le dossier complet de l'application en moins de deux minutes.

1. Commandes d'initialisation rapide

Dans votre terminal :

```

# 1. Créer le projet Expo React Native
npx create-expo-app BookmakerStudio --template blank-typescript
cd BookmakerStudio

# 2. Installer les icônes et la navigation
npx expo install @expo/vector-icons react-native-safe-area-context
npm install @react-navigation/native @react-navigation/bottom-tabs @react-navigation/native-stack
npx expo install react-native-screens

```

2. Arborescence des fichiers du projet

Créez l'organisation de dossiers suivante :

```

BookmakerStudio/
├── App.tsx
├── package.json
├── src/
│   ├── components/
│   │   ├── CustomTabBar.tsx
│   │   └── CutoutTicketCard.tsx
│   ├── screens/
│   │   ├── PopularScreen.tsx
│   │   ├── StudioTVScreen.tsx
│   │   ├── StudioCreationScreen.tsx
│   │   ├── PokerBetDetailScreen.tsx
│   │   ├── HistoryScreen.tsx
│   │   └── MenuScreen.tsx
│   └── utils/
│       └── pokerEngine.ts

```

3. Fichiers clés à copier-coller

App.tsx (Point d'entrée avec Navigation complète)

```

import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { Ionicons, MaterialCommunityIcons } from '@expo/vector-icons';

// Écrans
import StudioCreationScreen from './src/screens/StudioCreationScreen';

```

```

import StudioTVScreen from './src/screens/StudioTVScreen';
import PokerBetDetailScreen from './src/screens/PokerBetDetailScreen';

const Tab = createBottomTabNavigator();
const Stack = createNativeStackNavigator();

// Écran Populaire (Hub principal des studios)
function PopularScreen({ navigation }: any) {
  return (
    <View style={styles.container}>
      <Text style={styles.pageTitle}>Populaire</Text>

      <TouchableOpacity
        style={styles.menuCard}
        onPress={() => navigation.navigate('StudioCreation')}
      >
        <View style={styles.iconBox}>
          <MaterialCommunityIcons name="ticket-percent-outline" size={24} color="#2563eb" />
        </View>
        <View style={{ flex: 1 }}>
          <Text style={styles.cardTitle}>Accéder au studio</Text>
          <Text style={styles.cardSub}>Crée tes propres coupons de paris personnalisés.</Text>
        </View>
        <Ionicons name="chevron-forward" size={18} color="#94a3b8" />
      </TouchableOpacity>

      <TouchableOpacity
        style={styles.menuCard}
        onPress={() => navigation.navigate('StudioTV')}
      >
        <View style={styles.iconBox}>
          <Ionicons name="game-controller-outline" size={24} color="#2563eb" />
        </View>
        <View style={{ flex: 1 }}>
          <Text style={styles.cardTitle}>Accéder au studio TV</Text>
          <Text style={styles.cardSub}>Crée une manche TvBet Poker.</Text>
        </View>
        <Ionicons name="chevron-forward" size={18} color="#94a3b8" />
      </TouchableOpacity>
    </View>
  );
}

function PlaceholderScreen({ title }: { title: string }) {
  return (
    <View style={[styles.container, { justifyContent: 'center', alignItems: 'center' }]}>
      <Text style={{ fontSize: 16, color: '#64748b' }}>Écran {title}</Text>
    </View>
  );
}

// Barre d'onglets personnalisée avec le bouton central surélevé
function MainTabs({ navigation }: any) {
  return (
    <Tab.Navigator
      screenOptions={{
        headerShown: false,
        tabBarActiveTintColor: '#2563EB',
        tabBarInactiveTintColor: '#64748B',
        tabBarStyle: { height: 65, paddingBottom: 8 },
      }}
    >
      <Tab.Screen
        name="Populaire"
        component={PopularScreen}
        options={{
          tabBarIcon: ({ color, size }) => <Ionicons name="flame" size={size} color={color} />,
        }}
      />
      <Tab.Screen
        name="Favoris"
        children={() => <PlaceholderScreen title="Favoris" />}
        options={{
          tabBarIcon: ({ color, size }) => <Ionicons name="star" size={size} color={color} />,
        }}
      />
      <Tab.Screen
        name="Coupon"
        component={StudioCreationScreen}

```

```

    options={{
      tabBarButton: (props) => (
        <TouchableOpacity
          {...props}
          style={styles.floatingCenterBtn}
          onPress={() => navigation.navigate('StudioCreation')}
        >
          <MaterialCommunityIcons name="ticket-confirmation" size={26} color="#fff" />
        </TouchableOpacity>
      ),
    }}
  />
<Tab.Screen
  name="Historique"
  component={PokerBetDetailScreen}
  options={{
    tabBarIcon: ({ color, size }) => <Ionicons name="time" size={size} color={color} />,
  }}
/>
<Tab.Screen
  name="Menu"
  children={() => <PlaceholderScreen title="Menu Administration" />}
  options={{
    tabBarIcon: ({ color, size }) => <Ionicons name="grid" size={size} color={color} />,
  }}
/>
</Tab.Navigator>
);
}

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator screenOptions={{ headerShown: false }}>
        <Stack.Screen name="MainTabs" component={MainTabs} />
        <Stack.Screen name="StudioCreation" component={StudioCreationScreen} />
        <Stack.Screen name="StudioTV" component={StudioTVScreen} />
        <Stack.Screen name="PokerBetDetail" component={PokerBetDetailScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#F1F5F9',
    padding: 16,
    paddingTop: 50,
  },
  pageTitle: {
    fontSize: 20,
    fontWeight: '800',
    color: '#1E293B',
    marginBottom: 20,
    textAlign: 'center',
  },
  menuCard: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#fff',
    borderRadius: 14,
    padding: 16,
    marginBottom: 12,
    borderWidth: 1,
    borderColor: '#E2E8F0',
  },
  iconBox: {
    width: 44,
    height: 44,
    borderRadius: 12,
    backgroundColor: '#FFF6FF',
    justifyContent: 'center',
    alignItems: 'center',
    marginRight: 14,
  },
  cardTitle: {
    fontSize: 15,
    fontWeight: '700',

```

```

    color: '#1E293B',
  },
  cardSub: {
    fontSize: 12,
    color: '#64748B',
    marginTop: 2,
  },
  floatingCenterBtn: {
    top: -18,
    justifyContent: 'center',
    alignItems: 'center',
    width: 58,
    height: 58,
    borderRadius: 29,
    backgroundColor: '#1E40AF',
    shadowColor: '#1E40AF',
    shadowOffset: { width: 0, height: 6 },
    shadowOpacity: 0.35,
    shadowRadius: 5,
    elevation: 6,
  },
});

```

4. Lancer le projet

Dès que vous avez placé les fichiers dans les dossiers respectifs :

```
npx expo start
```

Scannez le QR Code avec **Expo Go** sur votre téléphone Android ou iPhone pour tester l'application directement en conditions réelles.

User prompt: Donne moi un rendu de mon app voir

Response: Voici un aperçu interactif complet qui reproduit fidèlement l'interface, les découpes de ticket et la navigation de votre application.

Vous pouvez tester ce prototype directement dans votre navigateur via [Expo Snack](#) (aucun téléchargement requis, compatible avec le visualiseur web ou l'application Expo Go sur votre téléphone) en y collant le code suivant :

Code complet du prototype interactif

```

import React, { useState } from 'react';
import {
  View,
  Text,
  StyleSheet,
  ScrollView,
  TouchableOpacity,
  TextInput,
  SafeAreaView,
  Switch,
  Modal,
} from 'react-native';
import { Ionicons, MaterialCommunityIcons } from '@expo/vector-icons';

export default function App() {
  // Navigation d'onglets
  const [activeTab, setActiveTab] = useState<'Populaire' | 'Favoris' | 'Studio' | 'Historique' | 'Menu'>('Historique');
  const [currentView, setCurrentView] = useState<'list' | 'detail'>('detail');
  const [isCouponWon, setIsCouponWon] = useState(false);
  const [actionSheetVisible, setActionSheetVisible] = useState(false);

  // État du Studio Custom
  const [stake, setStake] = useState('500');
  const [odd, setOdd] = useState('1.85');
  const [league, setLeague] = useState('Vietnam. V-League');
  const [enableCashout, setEnableCashout] = useState(false);

  // Rendu de l'écran Populaire
  const renderPopular = () => (
    <ScrollView style={styles.scrollPage} contentContainerStyle={{ padding: 16 }}>
      <Text style={styles.screenHeaderTitle}>Populaire</Text>

      {[
        { title: 'Accéder au studio', sub: 'Crée tes propres coupons de paris personnalisés.', icon: 'ticket-percent' },
        { title: 'Accéder à Apple of Fortune', sub: 'Mini-jeu : grimpe les paliers et encaisse avant la mauvaise pom', icon: 'apple' },
        { title: 'Accéder à Crash', sub: 'Mini-jeu : retire ta mise avant que l\'avion ne s\'envole trop loin.', icon: 'jet-fighter' },
        { title: 'Accéder au studio Mortal Kombat', sub: 'Crée des coupons de combats Mortal Kombat.', icon: 'sword' },
        { title: 'Accéder au studio TV', sub: 'Crée une manche TvBet Poker.', icon: 'television-play' },
      ]}
    </ScrollView>
  );

```

```

].map((item, idx) => (
  <TouchableOpacity
    key={idx}
    style={styles.cardItem}
    onPress={() => item.target === 'Studio' && setActiveTab('Studio')}
  >
    <View style={styles.iconCircle}>
      <MaterialCommunityIcons name={item.icon as any} size={22} color="#2563EB" />
    </View>
    <View style={{ flex: 1 }}>
      <Text style={styles.cardItemTitle}>{item.title}</Text>
      <Text style={styles.cardItemSub}>{item.sub}</Text>
    </View>
    <Ionicons name="chevron-forward" size={18} color="#94A3B8" />
  </TouchableOpacity>
))
</ScrollView>
);

// Rendu de l'écran Studio de création
const renderStudio = () => (
  <ScrollView style={styles.scrollPage} contentContainerStyle={{ padding: 16, paddingBottom: 60 }}>
    <Text style={styles.screenHeaderTitle}>Studio de création</Text>

    /* Banner */
    <View style={styles.banner}>
      <MaterialCommunityIcons name="ticket-percent" size={24} color="#2563EB" />
      <View style={{ flex: 1, marginLeft: 10 }}>
        <Text style={styles.bannerTitle}>Création guidée</Text>
        <Text style={styles.bannerSub}>ID, équipes, mise et résumé dans un seul parcours.</Text>
      </View>
    </View>

    /* Identité */
    <View style={styles.boxCard}>
      <Text style={styles.boxTitle}>IDENTITÉ</Text>
      <Text style={styles.fieldLabel}>ID personnalisé optionnel</Text>
      <TextInput style={styles.input} placeholder="Ex: 85517683766" />
      <Text style={styles.hint}>11 chiffres, commence par 85. Vide = généré automatiquement.</Text>
    </View>

    /* Événement */
    <View style={styles.boxCard}>
      <Text style={styles.boxTitle}>ÉVÉNEMENT</Text>
      <Text style={styles.fieldLabel}>Championnat</Text>
      <TextInput style={styles.input} value={league} onChangeText={setLeague} />

      <Text style={[styles.fieldLabel, { marginTop: 10 }]}>Cote</Text>
      <TextInput style={[styles.input, { width: 120 }]} value={odd} onChangeText={setOdd} keyboardType="numeric" />

      <TouchableOpacity
        style={[styles.btnPrimary, { marginTop: 14 }]}
        onPress={() => {
          setActiveTab('Historique');
          setCurrentView('detail');
        }}
      >
        <Text style={styles.btnPrimaryText}>+ Ajouter au coupon</Text>
      </TouchableOpacity>
    </View>

    /* Mise */
    <View style={styles.boxCard}>
      <Text style={styles.boxTitle}>MISE ET RÉSUMÉ</Text>
      <TextInput style={styles.input} value={stake} onChangeText={setStake} keyboardType="numeric" />

      <View style={styles.switchRow}>
        <Text style={styles.fieldLabel}>Activer la vente (cashout)</Text>
        <Switch value={enableCashout} onValueChange={setEnableCashout} />
      </View>
    </View>
  </ScrollView>
);

// Rendu de l'écran Détails du Coupon (Poker ou Custom)
const renderDetail = () => (
  <ScrollView style={styles.scrollPage} contentContainerStyle={{ padding: 16, paddingBottom: 60 }}>
    /* Header Interne */
    <View style={styles.ticketHeader}>

```

```

<TouchableOpacity onPress={() => setCurrentView('list')}>
  <Ionicons name="arrow-back" size={22} color="#1E293B" />
</TouchableOpacity>
<Text style={styles.headerBarTitle}>Détails du</Text>
<View style={{ flexDirection: 'row', gap: 12 }}>
  <TouchableOpacity onPress={() => setIsCouponWon(!isCouponWon)}>
    <Ionicons name={isCouponWon ? "checkmark-circle" : "sync"} size={22} color="#2563EB" />
  </TouchableOpacity>
  <TouchableOpacity onPress={() => setActionSheetVisible(true)}>
    <Ionicons name="ellipsis-vertical" size={22} color="#1E293B" />
  </TouchableOpacity>
</View>
</View>

{/* Billet Découpé */}
<View style={styles.ticket}>
  {/* En-tête Billet */}
  <View style={styles.ticketPadding}>
    <Text style={styles.ticketDate}>03.09.2026 (15:01)</Text>
    <Text style={styles.ticketNumber}>Combiné <Text style={{ fontWeight: 'normal' }}>№ 85517683766</Text></Tex

    <View style={styles.ticketDivider} />

    <View style={styles.ticketRow}>
      <View style={{ flexDirection: 'row', alignItems: 'center', gap: 6 }}>
        <Ionicons name="receipt-outline" size={16} color="#2563EB" />
        <Text style={styles.labelMuted}>Nombre d'événements : 2</Text>
      </View>
      <Text style={styles.labelMuted}>{isCouponWon ? '2 de 2 terminés' : '0 de 2 terminés'}</Text>
    </View>

    <View style={styles.ticketRow}>
      <Text style={styles.labelMuted}>Cote :</Text>
      <Text style={styles.valBold}>1000000</Text>
    </View>

    <View style={styles.ticketRow}>
      <Text style={styles.labelMuted}>Mise :</Text>
      <Text style={styles.valBold}>90 F</Text>
    </View>

    <View style={styles.ticketRow}>
      <Text style={styles.labelMuted}>{isCouponWon ? 'Gains : ' : 'Gains potentiels :'}</Text>
      <Text style={[styles.valBold, isCouponWon && { color: '#16A34A' }]}>9000000 F</Text>
    </View>

    <View style={styles.ticketRow}>
      <Text style={styles.labelMuted}>Statut :</Text>
      <View style={{ flexDirection: 'row', alignItems: 'center', gap: 4 }}>
        <Ionicons name="checkmark-circle" size={16} color={isCouponWon ? '#16A34A' : '#2563EB'} />
        <Text style={[styles.valBold, { color: isCouponWon ? '#16A34A' : '#2563EB' }]}>
          {isCouponWon ? 'Payé' : 'Accepté'}
        </Text>
      </View>
    </View>
  </View>

  {/* Découpe Perforée */}
  <View style={styles.cutoutLine}>
    <View style={[styles.notch, { left: -10 }]} />
    <View style={styles.dashedBorder} />
    <View style={[styles.notch, { right: -10 }]} />
  </View>

  {/* Détail Manche 1 */}
  <View style={styles.ticketPadding}>
    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
      <MaterialCommunityIcons name="gamepad-variant-outline" size={22} color="#2563EB" />
      <View>
        <Text style={styles.valBold}>TvBet. POKER</Text>
        <Text style={styles.ticketDate}>03.09.2026 (15:03)</Text>
      </View>
    </View>

    <View style={styles.pokerDrawBox}>
      <Text style={styles.pokeBadge}>POKE</Text>
      <Text style={styles.drawText}>Main 1: A♠, J♥</Text>
    </View>

  </View>

  <View style={styles.pokerDrawBox}>
    <Text style={styles.pokeBadge}>POKE</Text>
    <Text style={styles.drawText}>Main 1: A♠, J♥</Text>
  </View>

```

```

        <Text style={styles.drawText}>Main 2: 6♥, A♦</Text>
        <Text style={styles.drawText}>Main 3: 6♠, 10♣</Text>
        <Text style={styles.drawText}>Main 4: 3♠, 4♣</Text>
        <Text style={styles.drawText}>Main 5: J♦, 10♣</Text>
        <Text style={styles.drawText}>Main 6: Q♠, 4♦</Text>
        <Text style={[styles.drawText, { fontWeight: '700', marginTop: 4 }]}>Table : 2♣, 5♦, 7♠, 9♥, K♣
        <Text style={[styles.drawText, { fontWeight: '700' }]}>Gagnant : 1 (Carte haute)</Text>
    </View>
  ) : (
    <Text style={styles.pokeBadge}>POKE</Text>
  )}

  <View style={styles.ticketRow}>
    <Text style={{ fontSize: 13, fontWeight: '600', color: '#1E293B', flex: 1 }}>
      Combinaison gagnante. Carte haute
    </Text>
    <Text style={styles.valBold}>1000</Text>
  </View>

  <View style={styles.ticketRow}>
    <Text style={styles.labelMuted}>Statut :</Text>
    <Text style={[styles.valBold, { color: isCouponWon ? '#16A34A' : '#2563EB' }]}>
      {isCouponWon ? 'Gain' : 'Accepté'}
    </Text>
  </View>

  <View style={{ marginTop: 6 }}>
    <Text style={styles.labelMuted}>Informations supplémentaires :</Text>
    <Text style={styles.valBold}>PB840645</Text>
  </View>
</View>

  <TouchableOpacity style={styles.duplicateBtn}>
    <Text style={styles.duplicateBtnText}>DUPLIQUER LE COUPON DE PARI</Text>
  </TouchableOpacity>
</ScrollView>
);

// Rendu de la vue Historique Liste
const renderHistory = () => (
  <ScrollView style={styles.scrollPage} contentContainerStyle={{ padding: 16 }}>
    { /* Header Solde */ }
    <Text style={styles.screenHeaderTitle}>Historique des</Text>
    <View style={styles.balanceCard}>
      <View>
        <Text style={styles.labelMuted}>Compte principal</Text>
        <Text style={styles.balanceAmount}>182 419 282.08 F</Text>
      </View>
      <TouchableOpacity style={styles.depositBtn}>
        <Text style={styles.depositBtnText}>+ Effectuer un dépôt</Text>
      </TouchableOpacity>
    </View>

    { /* Billet Résumé cliquable */ }
    <TouchableOpacity style={styles.historyCard} onPress={() => setCurrentView('detail')}>
      <View style={styles.ticketRow}>
        <Text style={styles.ticketDate}>03.09.2026 (15:01) · N° 85517683766</Text>
        <Ionicons name="ellipsis-horizontal" size={20} color="#64748B" />
      </View>
      <Text style={styles.valBold}>Combiné 2</Text>

      <View style={styles.ticketDivider} />

      <View style={styles.ticketRow}>
        <Text style={styles.labelMuted}>Cote :</Text>
        <Text style={styles.valBold}>1000000</Text>
      </View>
      <View style={styles.ticketRow}>
        <Text style={styles.labelMuted}>Mise :</Text>
        <Text style={styles.valBold}>90 F</Text>
      </View>
      <View style={styles.ticketRow}>
        <Text style={styles.labelMuted}>{isCouponWon ? 'Gains :' : 'Gains potentiels :'}</Text>
        <Text style={[styles.valBold, isCouponWon && { color: '#16A34A' }]}>9000000 F</Text>
      </View>
      <View style={styles.ticketRow}>
        <Text style={styles.labelMuted}>Statut :</Text>
        <Text style={[styles.valBold, { color: isCouponWon ? '#16A34A' : '#2563EB' }]}>

```

```

        {isCouponWon ? 'Payé' : 'Accepté'}
      </Text>
    </View>
  </TouchableOpacity>
</ScrollView>
);

return (
  <SafeAreaView style={styles.mainContainer}>
    { /* Contenu Actif */ }
    <View style={{ flex: 1 }}>
      {activeTab === 'Populaire' && renderPopular()}
      {activeTab === 'Studio' && renderStudio()}
      {activeTab === 'Historique' && (currentView === 'detail' ? renderDetail() : renderHistory())}
      {(activeTab === 'Favoris' || activeTab === 'Menu') && (
        <View style={[styles.scrollPage, { justifyContent: 'center', alignItems: 'center' }]}>
          <Text style={styles.labelMuted}>Module {activeTab} disponible</Text>
        </View>
      )}
    </View>

    { /* Modal Bottom Sheet Actions (...) */ }
    <Modal visible={actionSheetVisible} transparent animationType="slide">
      <TouchableOpacity
        style={styles.modalOverlay}
        activeOpacity={1}
        onPress={() => setActionSheetVisible(false)}
      >
        <View style={styles.sheetContainer}>
          <View style={styles.sheetHandle} />
          <Text style={styles.sheetTitle}>Options du pari</Text>

          {[
            { label: 'Voir le détail', icon: 'ticket-outline', action: () => setActionSheetVisible(false) },
            {
              label: isCouponWon ? 'Passer en Accepté' : 'Saisir les scores (Valider Gain)',
              icon: 'pencil-outline',
              action: () => { setIsCouponWon(!isCouponWon); setActionSheetVisible(false); }
            },
            { label: 'Définir la vente', icon: 'pricetag-outline', action: () => setActionSheetVisible(false) },
            { label: 'Partager le coupon', icon: 'share-outline', action: () => setActionSheetVisible(false) },
            { label: 'Supprimer de l\'historique', icon: 'trash-outline', color: '#EF4444', action: () => setActio
          ].map((act, i) => (
            <TouchableOpacity key={i} style={styles.sheetItem} onPress={act.action}>
              <Icons name={act.icon as any} size={20} color={act.color || '#2563EB'} />
              <Text style={[styles.sheetItemText, act.color ? { color: act.color } : null]}>{act.label}</Text>
              <Icons name="chevron-forward" size={16} color="#94A3B8" />
            </TouchableOpacity>
          ))}
        </View>
      </TouchableOpacity>
    </Modal>

    { /* Bottom Bar avec bouton central surélevé */ }
    <View style={styles.bottomBar}>
      <TouchableOpacity style={styles.tabBtn} onPress={() => setActiveTab('Populaire')}>
        <Icons name="flame" size={22} color={activeTab === 'Populaire' ? '#2563EB' : '#94A3B8'} />
        <Text style={[styles.tabLabel, activeTab === 'Populaire' && styles.tabActive]}>Populaire</Text>
      </TouchableOpacity>

      <TouchableOpacity style={styles.tabBtn} onPress={() => setActiveTab('Favoris')}>
        <Icons name="star" size={22} color={activeTab === 'Favoris' ? '#2563EB' : '#94A3B8'} />
        <Text style={[styles.tabLabel, activeTab === 'Favoris' && styles.tabActive]}>Favoris</Text>
      </TouchableOpacity>

      { /* Bouton Flottant Coupon */ }
      <TouchableOpacity
        style={styles.floatingCenter}
        onPress={() => {
          setActiveTab('Studio');
        }}
      >
        <MaterialCommunityIcons name="ticket-confirmation" size={26} color="fff" />
      </TouchableOpacity>

      <TouchableOpacity style={styles.tabBtn} onPress={() => { setActiveTab('Historique'); setCurrentView('list'); }}>
        <Icons name="time" size={22} color={activeTab === 'Historique' ? '#2563EB' : '#94A3B8'} />
        <Text style={[styles.tabLabel, activeTab === 'Historique' && styles.tabActive]}>Historique</Text>
      </TouchableOpacity>
    </View>
  </SafeAreaView>
);

```

```

    <TouchableOpacity style={styles.tabBtn} onPress={() => setActiveTab('Menu')}>
      <Icons name="grid" size={22} color={activeTab === 'Menu' ? '#2563EB' : '#94A3B8'} />
      <Text style={[[styles.tabLabel, activeTab === 'Menu' && styles.tabActive]]>Menu</Text>
    </TouchableOpacity>
  </View>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  mainContainer: { flex: 1, backgroundColor: '#F1F5F8' },
  scrollPage: { flex: 1 },
  screenHeightTitle: { fontSize: 20, fontWeight: '800', color: '#1E293B', textAlign: 'center', marginVertical: 12 },

  // Cartes Populaire
  cardItem: { flexDirection: 'row', alignItems: 'center', backgroundColor: '#fff', padding: 14, borderRadius: 14, margin: 10 },
  iconCircle: { width: 40, height: 40, borderRadius: 10, backgroundColor: '#EFF6FF', alignItems: 'center', justifyCo
  cardItemTitle: { fontSize: 14, fontWeight: '700', color: '#1E293B' },
  cardItemSub: { fontSize: 12, color: '#64748B', marginTop: 2 },

  // Studio
  banner: { flexDirection: 'row', alignItems: 'center', backgroundColor: '#DBEAFE', padding: 12, borderRadius: 12, margin: 10 },
  bannerTitle: { fontSize: 14, fontWeight: '700', color: '#1E40AF' },
  bannerSub: { fontSize: 11, color: '#1E40AF' },
  boxCard: { backgroundColor: '#fff', padding: 14, borderRadius: 14, margin: 10, borderWidth: 1, borderColor: '#F1F5F8' },
  boxTitle: { fontSize: 13, fontWeight: '800', color: '#1E40AF', margin: 10 },
  fieldLabel: { fontSize: 12, fontWeight: '600', color: '#64748B', margin: 10 },
  input: { backgroundColor: '#F8F9FA', borderRadius: 8, borderWidth: 1, borderColor: '#CBD5E1', padding: 10, fontSiz
  hint: { fontSize: 10, color: '#94A3B8', margin: 10 },
  switchRow: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', margin: 10 },
  btnPrimary: { backgroundColor: '#1E40AF', padding: 12, borderRadius: 8, alignItems: 'center' },
  btnPrimaryText: { color: '#fff', fontWeight: '700' },

  // Historique & Ticket
  balanceCard: { backgroundColor: '#fff', padding: 16, borderRadius: 14, flexDirection: 'row', justifyContent: 'space-between', margin: 10 },
  balanceAmount: { fontSize: 18, fontWeight: '800', color: '#1E293B', margin: 10 },
  depositBtn: { backgroundColor: '#1E40AF', padding: 8, paddingHorizontal: 12, borderRadius: 8 },
  depositBtnText: { color: '#fff', fontWeight: '700', fontSize: 12 },
  historyCard: { backgroundColor: '#fff', borderRadius: 14, padding: 14, margin: 10, borderWidth: 1, borderColor: '#F1F5F8' },

  ticketHeader: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', margin: 10 },
  headerBarTitle: { fontSize: 16, fontWeight: '700', color: '#1E293B' },
  ticket: { backgroundColor: '#fff', borderRadius: 16, overflow: 'hidden', margin: 10, borderWidth: 1, borderColor: '#E2E8F0' },
  ticketPadding: { padding: 16 },
  ticketDate: { fontSize: 11, color: '#64748B' },
  ticketNumber: { fontSize: 16, fontWeight: '800', color: '#1E293B', margin: 10 },
  ticketDivider: { height: 1, backgroundColor: '#F1F5F9', margin: 10 },
  ticketRow: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', margin: 10 },
  labelMuted: { fontSize: 13, color: '#64748B' },
  valBold: { fontSize: 14, fontWeight: '700', color: '#1E293B' },

  // Encoches de ticket
  cutoutLine: { height: 20, backgroundColor: '#fff', position: 'relative', justifyContent: 'center' },
  notch: { width: 20, height: 20, borderRadius: 10, backgroundColor: '#F1F5F8', position: 'absolute', top: 0 },
  dashedBorder: { borderBottomWidth: 1, borderColor: '#CBD5E1', borderStyle: 'dashed', marginHorizontal: 16 },

  pokerDrawBox: { alignItems: 'center', margin: 10 },
  pokeBadge: { textAlign: 'center', fontWeight: '800', color: '#475569', fontSize: 12, letterSpacing: 1, margin: 10 },
  drawText: { fontSize: 12, color: '#334155', lineHeight: 1.2 },

  duplicateBtn: { backgroundColor: '#DBEAFE', borderRadius: 12, padding: 16, alignItems: 'center', margin: 10 },
  duplicateBtnText: { color: '#2563EB', fontWeight: '800', fontSize: 13 },

  // Bottom Sheet Modal
  modalOverlay: { flex: 1, backgroundColor: 'rgba(0,0,0,0.4)', justifyContent: 'flex-end' },
  sheetContainer: { backgroundColor: '#fff', borderTopLeftRadius: 20, borderTopRightRadius: 20, padding: 20, margin: 10 },
  sheetHandle: { width: 40, height: 4, backgroundColor: '#CBD5E1', borderRadius: 2, alignSelf: 'center', margin: 10 },
  sheetTitle: { fontSize: 16, fontWeight: '800', textAlign: 'center', margin: 10 },
  sheetItem: { flexDirection: 'row', alignItems: 'center', padding: 12, gap: 12 },
  sheetItemText: { fontSize: 14, fontWeight: '600', color: '#1E293B', flex: 1 },

  // Bottom Navigation Bar
  bottomBar: { flexDirection: 'row', height: 62, backgroundColor: '#fff', borderTopWidth: 1, borderTopColor: '#E2E8F0' },
  tabBtn: { alignItems: 'center', justifyContent: 'center' },
  tabLabel: { fontSize: 10, color: '#94A3B8', margin: 10 },
  tabActive: { color: '#2563EB', fontWeight: '700' },
  floatingCenter: { top: -16, width: 54, height: 54, borderRadius: 27, backgroundColor: '#1E40AF', alignItems: 'center', justifyCo
});

```

Ce que ce prototype vous permet de tester immédiatement :

1. **Bascule Gagné / Accepté** : Cliquez sur l'icône de synchronisation en haut à droite du détail pour voir le coupon passer instantanément de l'état bleu *Accepté* à l'état vert *Payé / Gain* avec tout le tableau de cartes (flop/turn/river et mains 1 à 6).
 2. **Options du pari (•••)** : Ouvrez le volet inférieur contextuel (bottom sheet) pour changer le score ou simuler les actions.
 3. **Le bouton flottant central** : Il vous ramène directement au formulaire du Studio de création.
-